CS 575 Lab Notebook

Lab 7

Name: Jarren Calizo
OdinID: calizo
Date: February 22, 2026

Table of Contents

Lab 7.1 Command and Configuration Generation 3
Step 1. Google dork commands 3
1. Provide a table for your lab notebook of the prompts used for each 3
Step 2. jq JSON parsing commands 4
1. Take a screenshot of the result that includes your OdinId 4
2. Take a screenshot of the result that includes your OdinId 5
3. Take a screenshot of the result that includes your OdinId 6
Step 3. iptables commands 7
1. Take a screenshot of the result that includes your OdinID 7
2. Take a screenshot of the prompt and result that includes your OdinID 7
Step 4. Google Cloud commands 8
1. Take a screenshot of the prompt and result that includes your OdinID 8
2. Take a screenshot of the prompt and result that includes your OdinID 9
Step 5. nginx configuration 11
1. Take a screenshot of the result that includes your OdinID 11
2. Take a screenshot of the prompt and result that includes your OdinID 12
Step 6. Terraform configuration 14
1. Take a screenshot of the result that includes your OdinID 14
2. Take a screenshot of the prompt and result that includes your OdinID 15
Step 7. Docker configuration 17
1. Take a screenshot of the result that includes your OdinID 17
2. Take a screenshot of the prompt and result that includes your OdinID 18
Step 8. Kubernetes configuration 21
1. Take a screenshot of the result that includes your OdinID 21
2. Take a screenshot of the prompt and result that includes your OdinID 22
Lab 7.2 Command Agents 24
Step 1. Linux processes 24
1. Include a table of successful commands generated for each task for your notebook 24
Step 2. Linux files and regular expressions (grep) 25
1. Include a table of successful commands generated for each task for your notebook 25
Step 3. Linux file searching (find) 26
1. Include a table of successful commands generated for each task for your notebook 26
Step 4. Linux firewall 26
1. Include a table of successful commands generated for each task for your

notebook 26
Step 5. Linux audit logs 27
1. Include a table of successful commands generated for each task for your notebook 27
Step 6. tcpdump 28
1. Take a screenshot showing the results for each that includes your OdinId 28
Step 7. Linux package vulnerabilities 31
1. Include a table of successful commands generated for each task for your notebook 31
Step 8. Linux package vulnerabilities (2) 32
1. Which vulnerabilities did the custom tools discover that weren't found previously? 32

Lab 7.1 Command and Configuration Generation

Step 1. Google dork commands

1. Provide a table for your lab notebook of the prompts used for each

Target	Prompt To LLM	LLM Output	Does it match the codelab?
Virtual desktop endpoints	Generate the exact Google dork to find VNC web clients by searching for the phrase VNC Desktop and requiring the URL to include port 5800. Output only the dork	"VNC Desktop" inurl:5800	Yes
Restricted directories and paths (robots)	Generate the exact Google dork that finds robots.txt or robot.txt pages where the text contains disallow, restricted to txt results. Output only the dork.	(inurl:robots.txt OR inurl:robot.txt) intext:disallow filetype:txt	Yes (similarly)
Exposed database administration interface	Generate the exact Google dork to find phpMyAdmin pages, restricted to the pdx.edu domain wildcard. Output only the dork.	site:*.pdx.edu inurl:phpmyadmin	Yes (similarly)
Unprotected database backups	Generate the exact Google dork to find directory listings containing database.sql.zip. Output only the	intitle:"index of" "database.sql.zip"	Yes

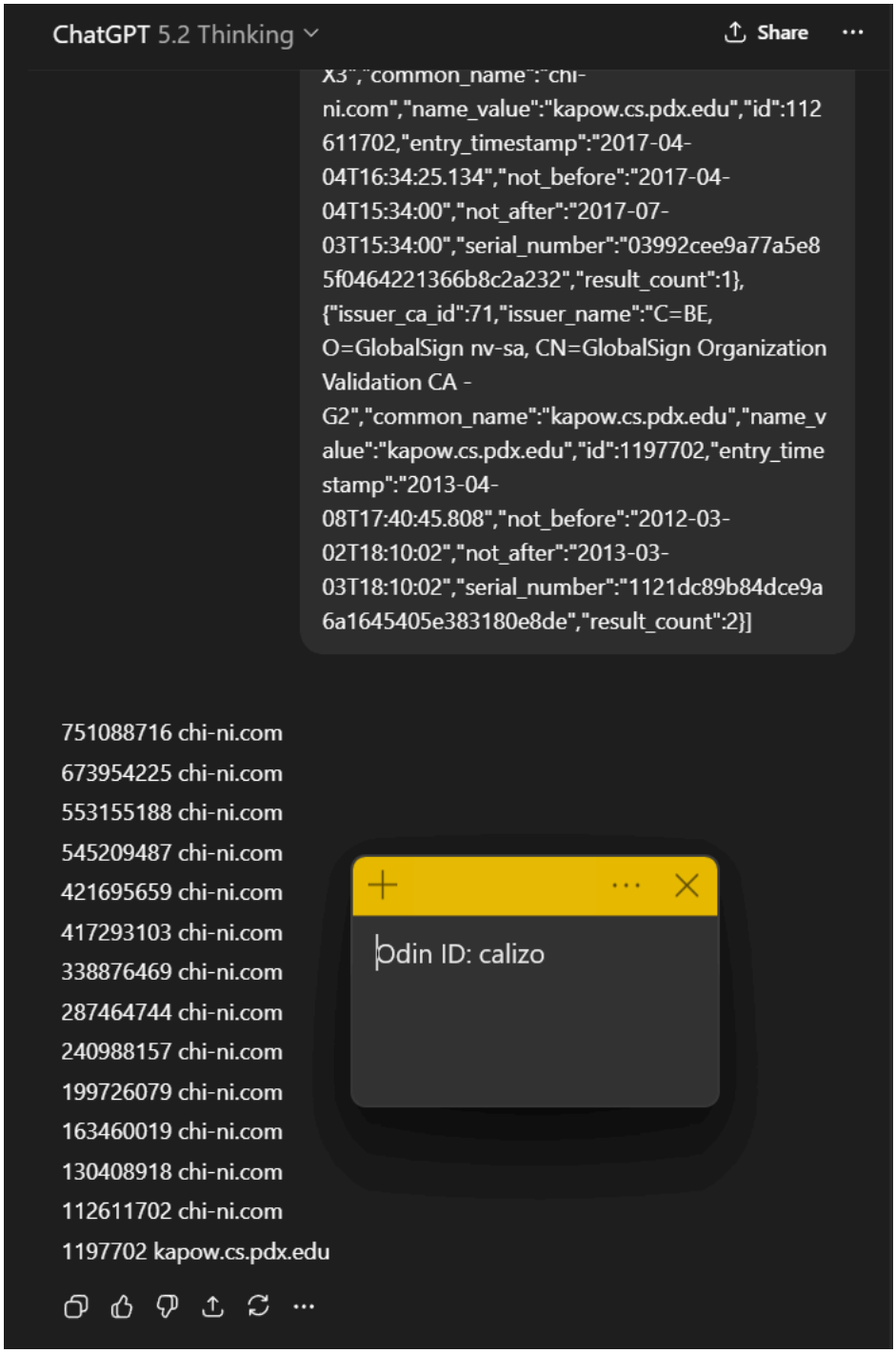
	dork.		
Credentials	Generate the exact Google dork to find directory listings containing passwords. Output only the dork.	intitle:"index of" "passwords"	Yes (similarly)
Secret files	Generate the exact Google dork to find PDFs containing the phrase Not for Public Release. Output only the dork	"Not for Public Release" filetype:pdf	Yes and no (if you consider them filetype & ext the same, then yes)

Step 2. jq JSON parsing commands

1. Take a screenshot of the result that includes your OdinId



2. Take a screenshot of the result that includes your OdinId



3. Take a screenshot of the result that includes your OdinId

```
┌──(calizo㉿kali-vm)-[~/lab7_1]
└─$ cutoff="$(date -u -d '1 year ago' +%Y-%m-%dT%H:%M:%S)"

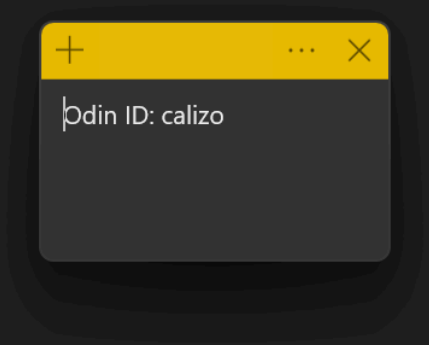
┌──(calizo㉿kali-vm)-[~/lab7_1]
└─$ echo "Cutoff UTC: $cutoff"
Cutoff UTC: 2025-02-23T05:59:22

┌──(calizo㉿kali-vm)-[~/lab7_1]
└─$ jq -r --arg cutoff "$cutoff" '
  .[]
  | select(.entry_timestamp[0:19] >= $cutoff)
  | .name_value
  | split("\n")[]
  | ascii_downcase
  | sub("^\\*\\."; "")
  | select(length > 0)
' cs_pdx.json \
| LC_ALL=C sort -u \
> cs_pdx_domains_last_year.txt

┌──(calizo㉿kali-vm)-[~/lab7_1]
└─$ wc -l cs_pdx_domains_last_year.txt
20 cs_pdx_domains_last_year.txt

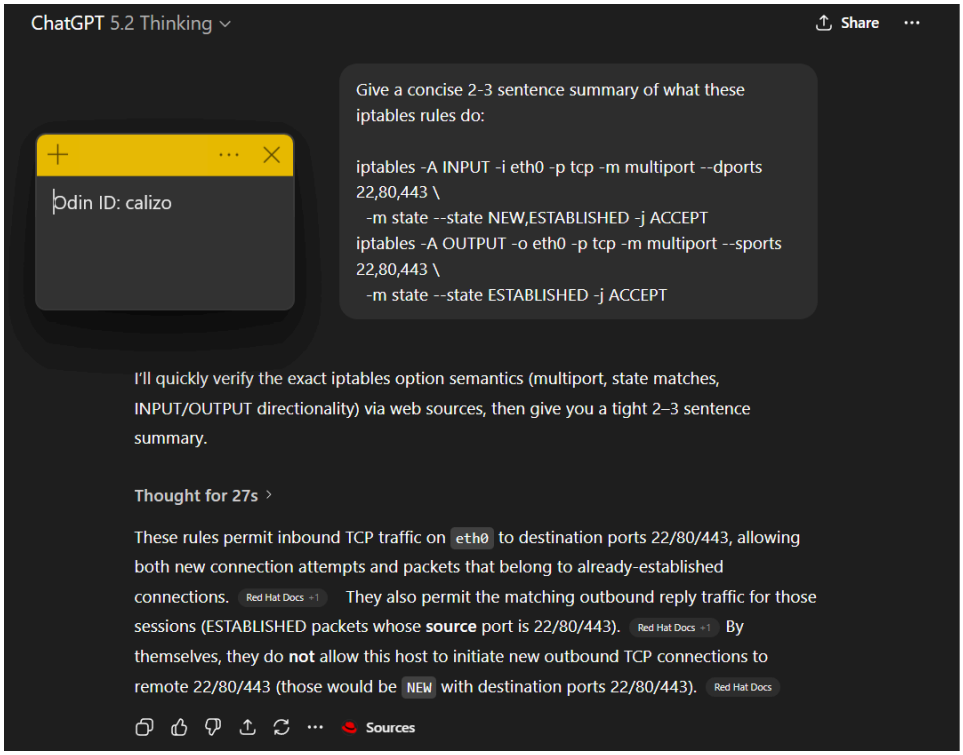
┌──(calizo㉿kali-vm)-[~/lab7_1]
└─$ cat cs_pdx_domains_last_year.txt
ada.cs.pdx.edu
busdata.cs.pdx.edu
cecs.pdx.edu
codelabs.cs.pdx.edu
cs.pdx.edu
cybersecurity.cs.pdx.edu
dbase.cecs.pdx.edu
dbclass.cs.pdx.edu
epl.cs.pdx.edu
feedback.cs.pdx.edu
mashimaro.cs.pdx.edu
piny.svcs.cs.pdx.edu
psas.cs.pdx.edu
secure.piny.svcs.cs.pdx.edu
stemrobotics.cs.pdx.edu
svcs.cs.pdx.edu
web.cs.pdx.edu
wics.cs.pdx.edu
wiki.cs.pdx.edu
zulip.cs.pdx.edu

┌──(calizo㉿kali-vm)-[~/lab7_1]
└─$ ls
```

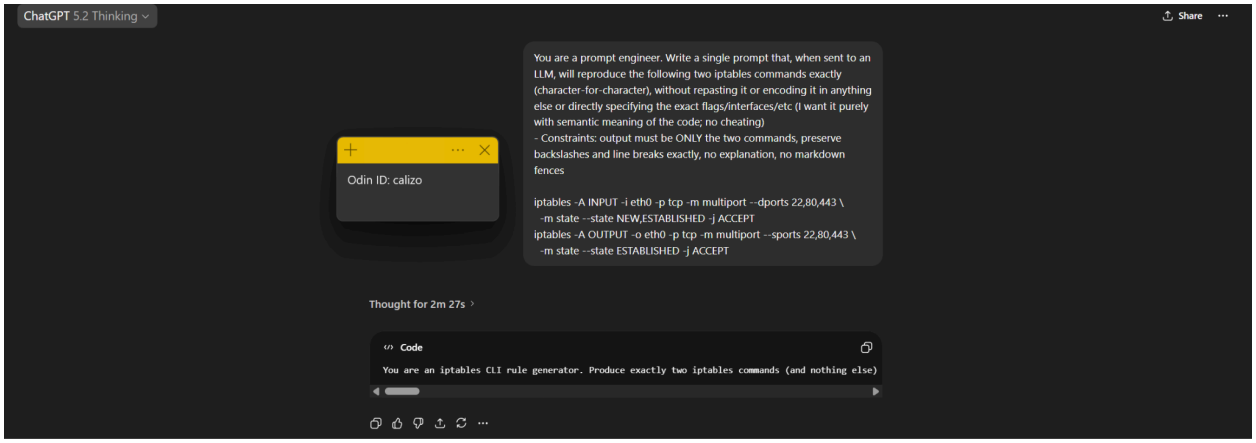


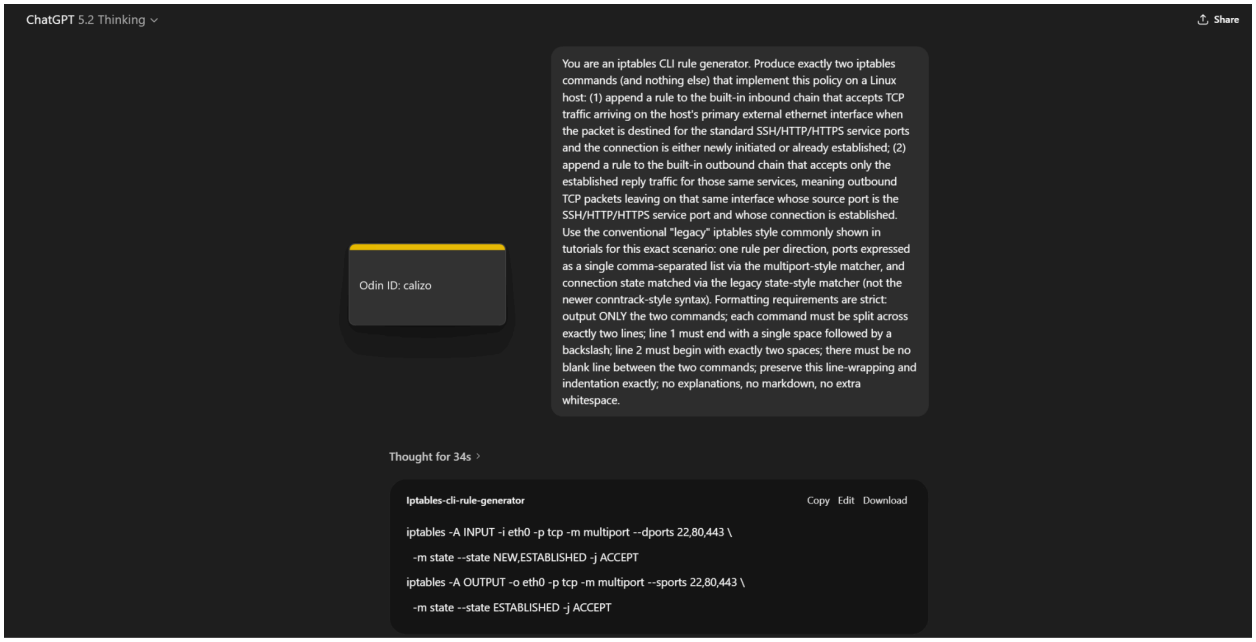
Step 3. iptables commands

1. Take a screenshot of the result that includes your OdinID



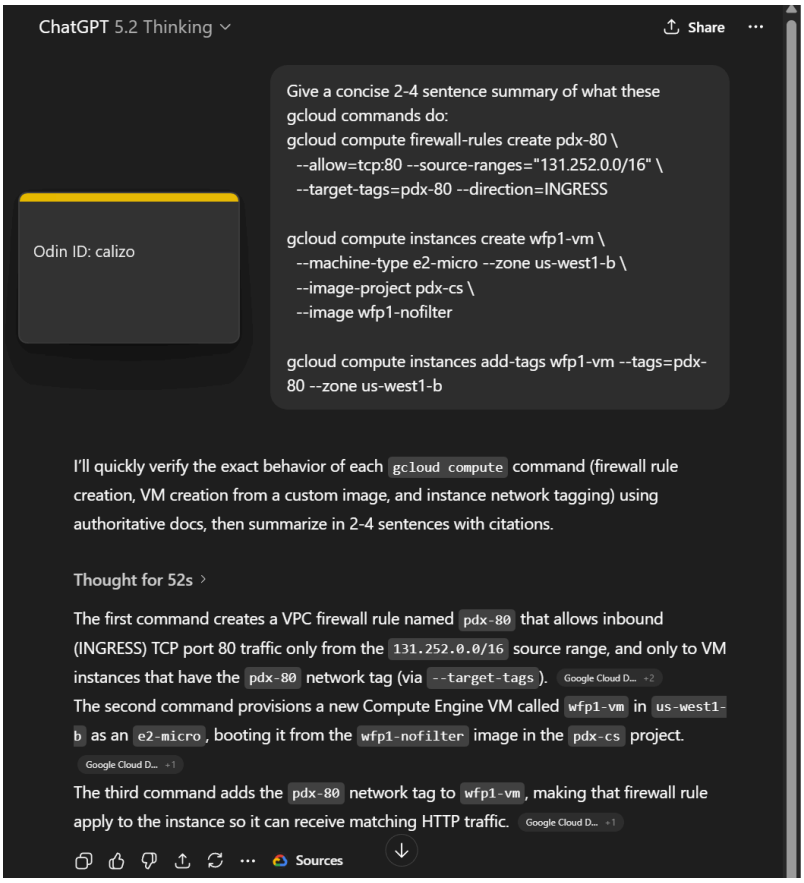
2. Take a screenshot of the prompt and result that includes your OdinID



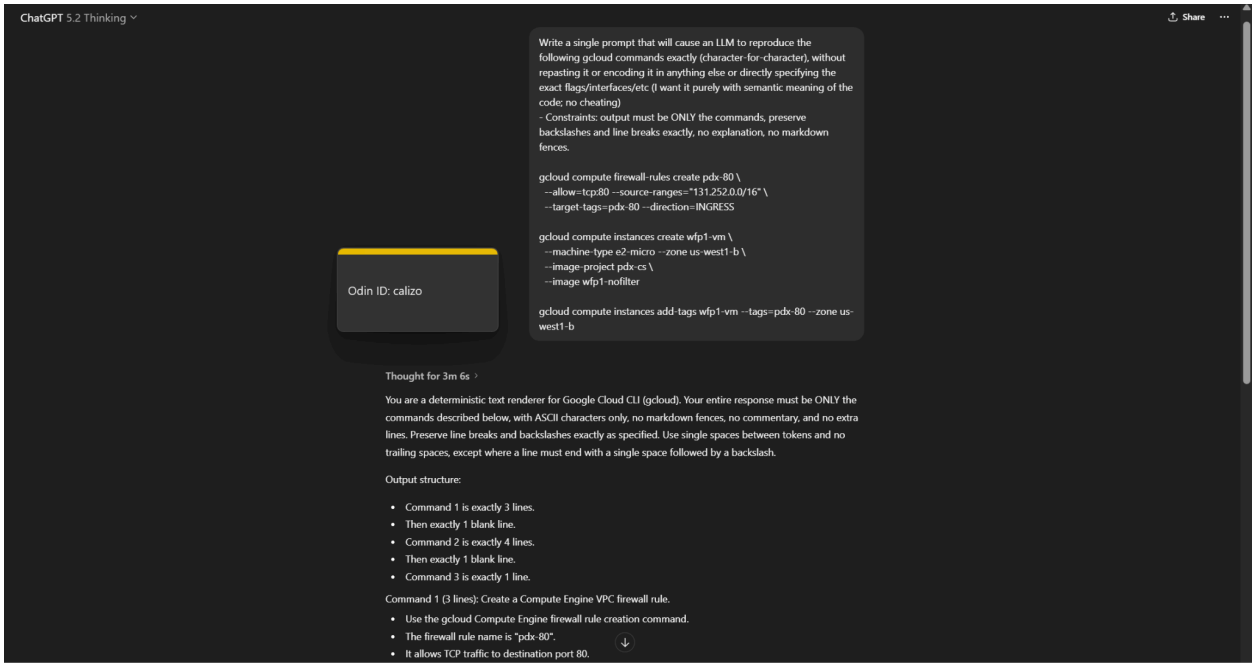


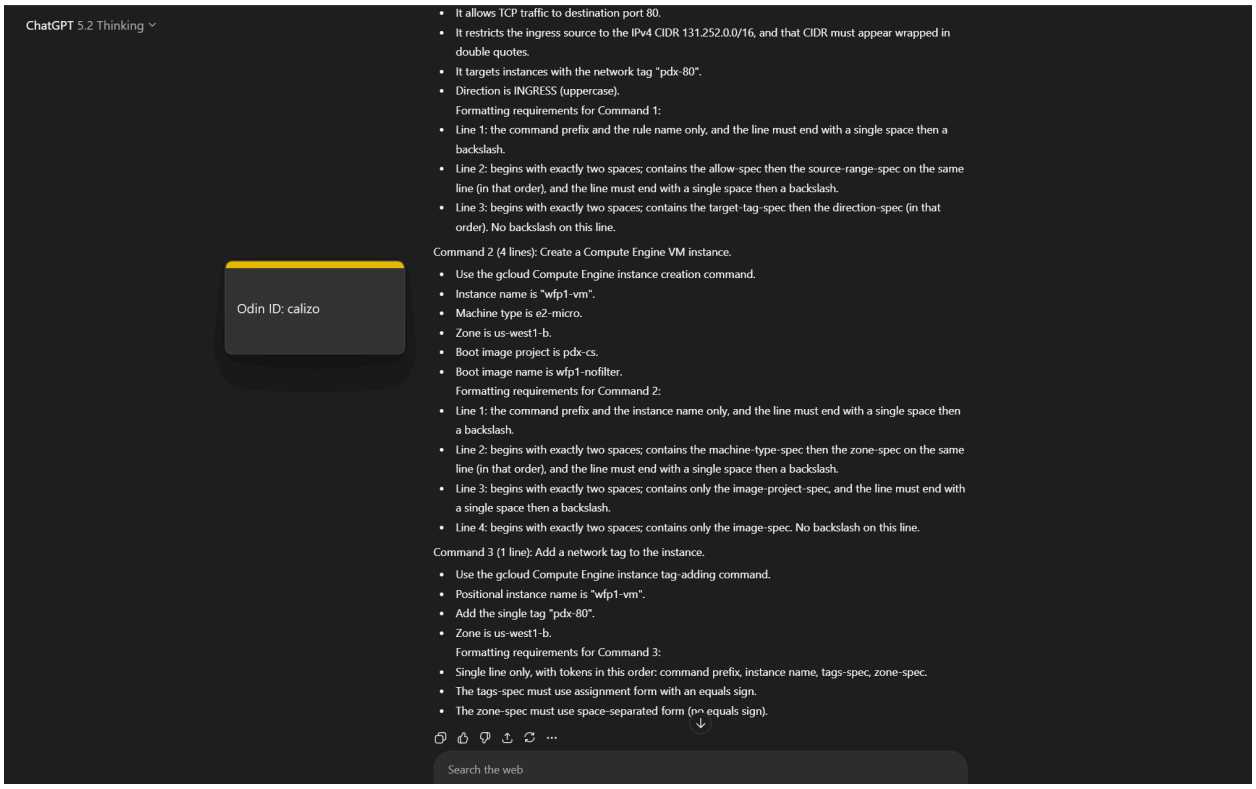
Step 4. Google Cloud commands

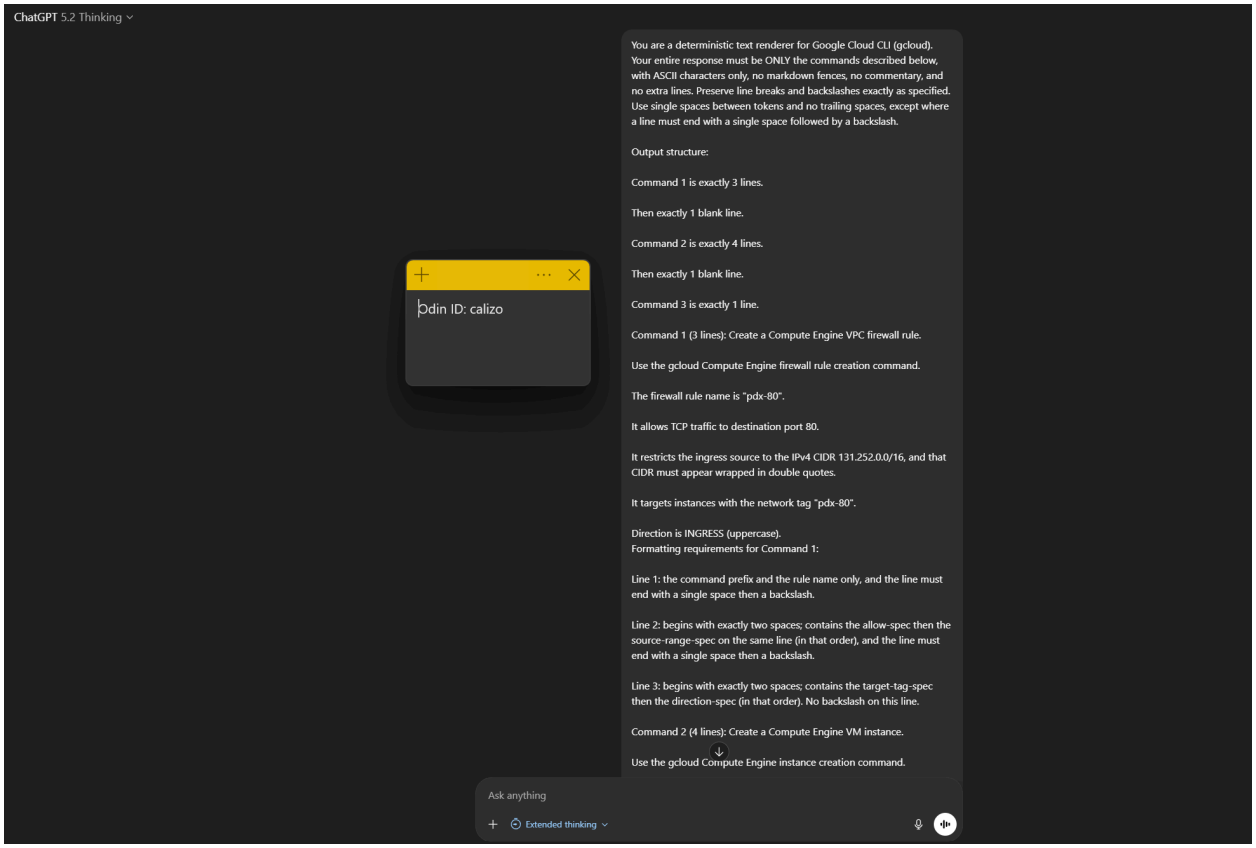
1. Take a screenshot of the prompt and result that includes your OdinID

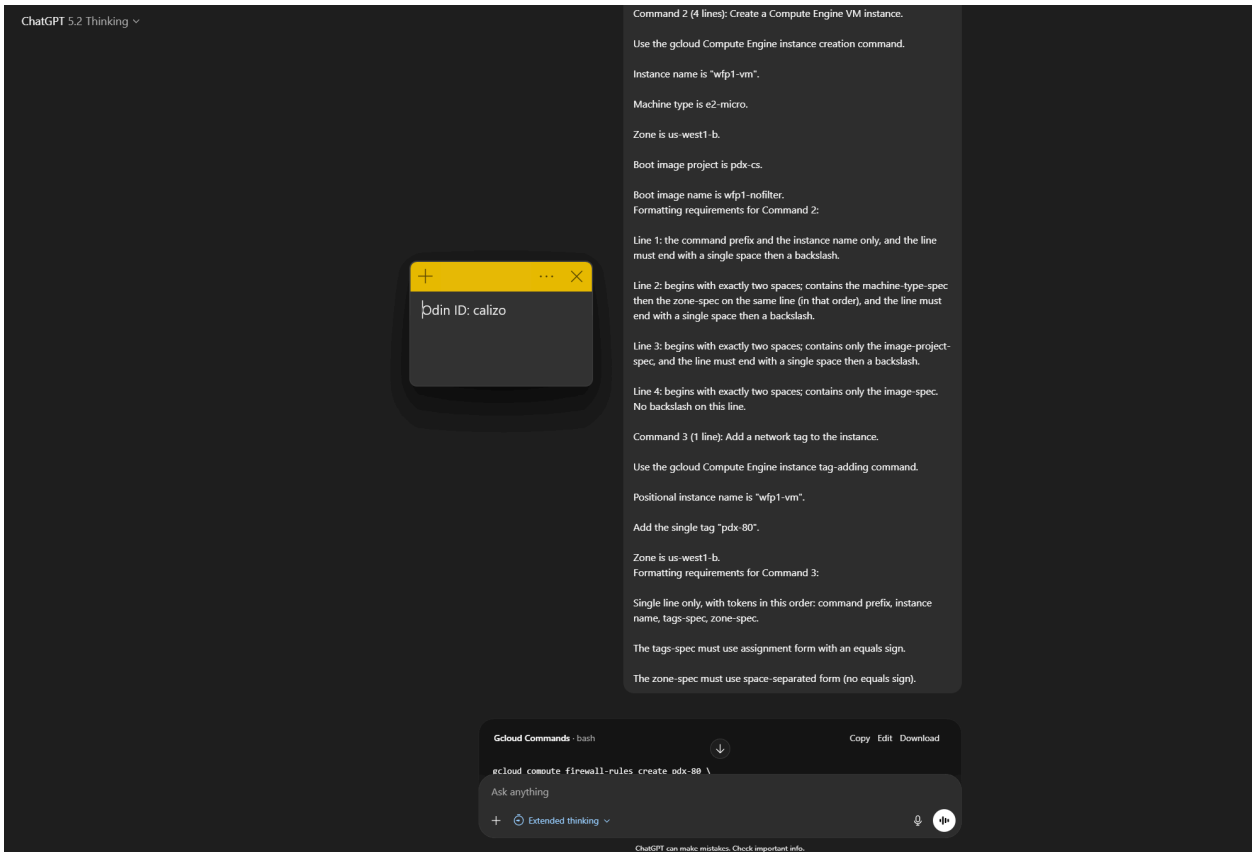


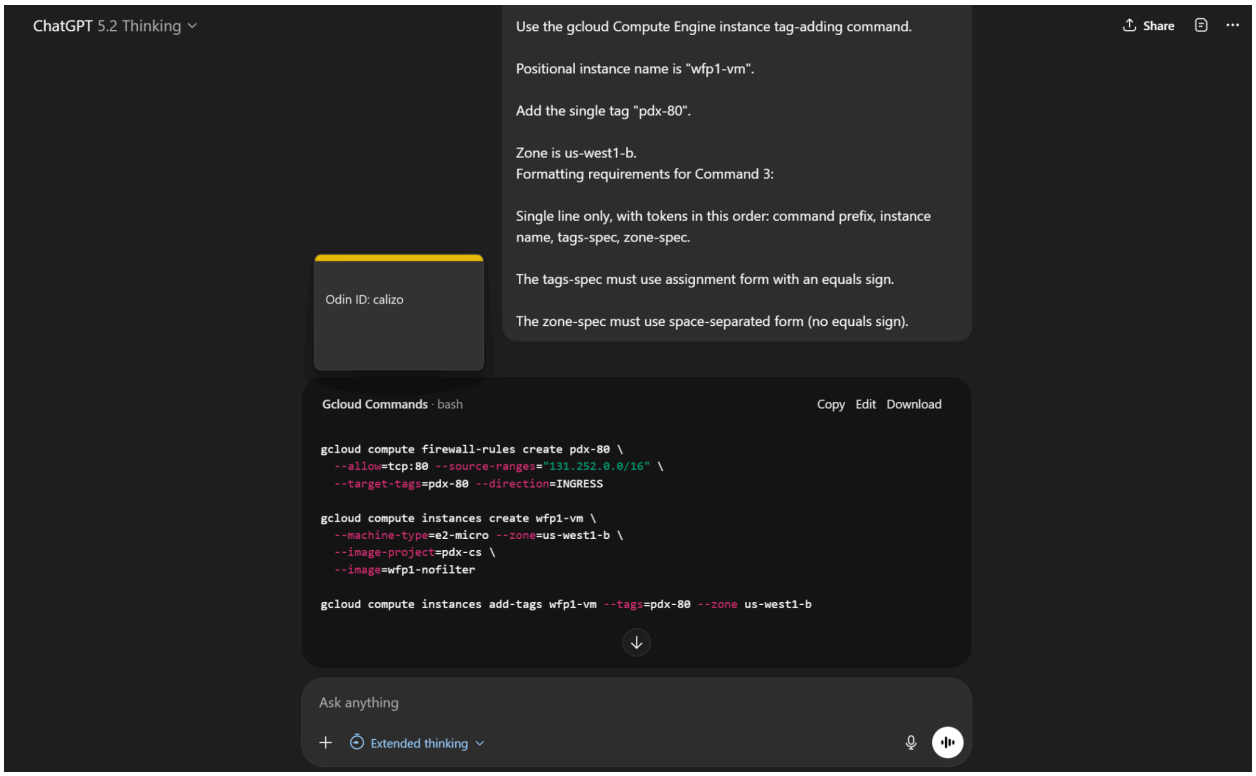
2. Take a screenshot of the prompt and result that includes your OdinID





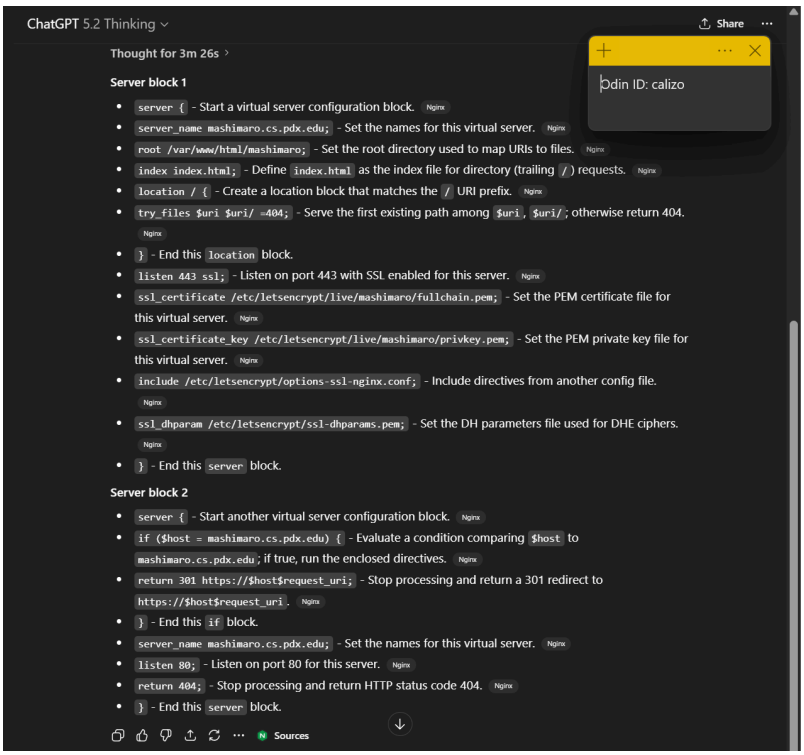

ChatGPT 5.2 Thinking
You are a deterministic text renderer for Google Cloud CLI (gcloud). Your entire response must be ONLY the commands described below, with ASCII characters only, no markdown fences, no commentary, and no extra lines. Preserve line breaks and backslashes exactly as specified. Use single spaces between tokens and no trailing spaces, except where a line must end with a single space followed by a backslash.
Output structure:
Command 1 is exactly 3 lines.
Then exactly 1 blank line.
Command 2 is exactly 4 lines.
Then exactly 1 blank line.
Command 3 is exactly 1 line.
Command 1 (3 lines): Create a Compute Engine VPC firewall rule.
Use the gcloud Compute Engine firewall rule creation command.
The firewall rule name is "pdx-80".
It allows TCP traffic to destination port 80.
It restricts the ingress source to the IPv4 CIDR 131.252.0.0/16, and that CIDR must appear wrapped in double quotes.
It targets instances with the network tag "pdx-80".
Direction is INGRESS (uppercase).
Formatting requirements for Command 1:
Line 1: the command prefix and the rule name only, and the line must end with a single space then a backslash.
Line 2: begins with exactly two spaces; contains the allow-spec then the source-range-spec on the same line (in that order), and the line must end with a single space then a backslash.
Line 3: begins with exactly two spaces; contains the target-tag-spec then the direction-spec (in that order). No backslash on this line.
Command 2 (4 lines): Create a Compute Engine VM instance.
Use the gcloud Compute Engine instance creation command.
pdin ID: calizo
Ask anything
Extended thinking


ChatGPT 5.2 Thinking
Command 2 (4 lines): Create a Compute Engine VM instance.
Use the gcloud Compute Engine instance creation command.
Instance name is "wfp1-vm".
Machine type is e2-micro.
Zone is us-west1-b.
Boot image project is pdx-cs.
Boot image name is wfp1-nofilter.
Formatting requirements for Command 2:
Line 1: the command prefix and the instance name only, and the line must end with a single space then a backslash.
Line 2: begins with exactly two spaces; contains the machine-type-spec then the zone-spec on the same line (in that order), and the line must end with a single space then a backslash.
Line 3: begins with exactly two spaces; contains only the image-project-spec, and the line must end with a single space then a backslash.
Line 4: begins with exactly two spaces; contains only the image-spec. No backslash on this line.
Command 3 (1 line): Add a network tag to the instance.
Use the gcloud Compute Engine instance tag-adding command.
Positional instance name is "wfp1-vm".
Add the single tag "pdx-80".
Zone is us-west1-b.
Formatting requirements for Command 3:
Single line only, with tokens in this order: command prefix, instance name, tags-spec, zone-spec.
The tags-spec must use assignment form with an equals sign.
The zone-spec must use space-separated form (no equals sign).
pdin ID: calizo
Gcloud Commands - bash
Copy Edit Download
gcloud compute firewall-rules create pdx-80 \
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

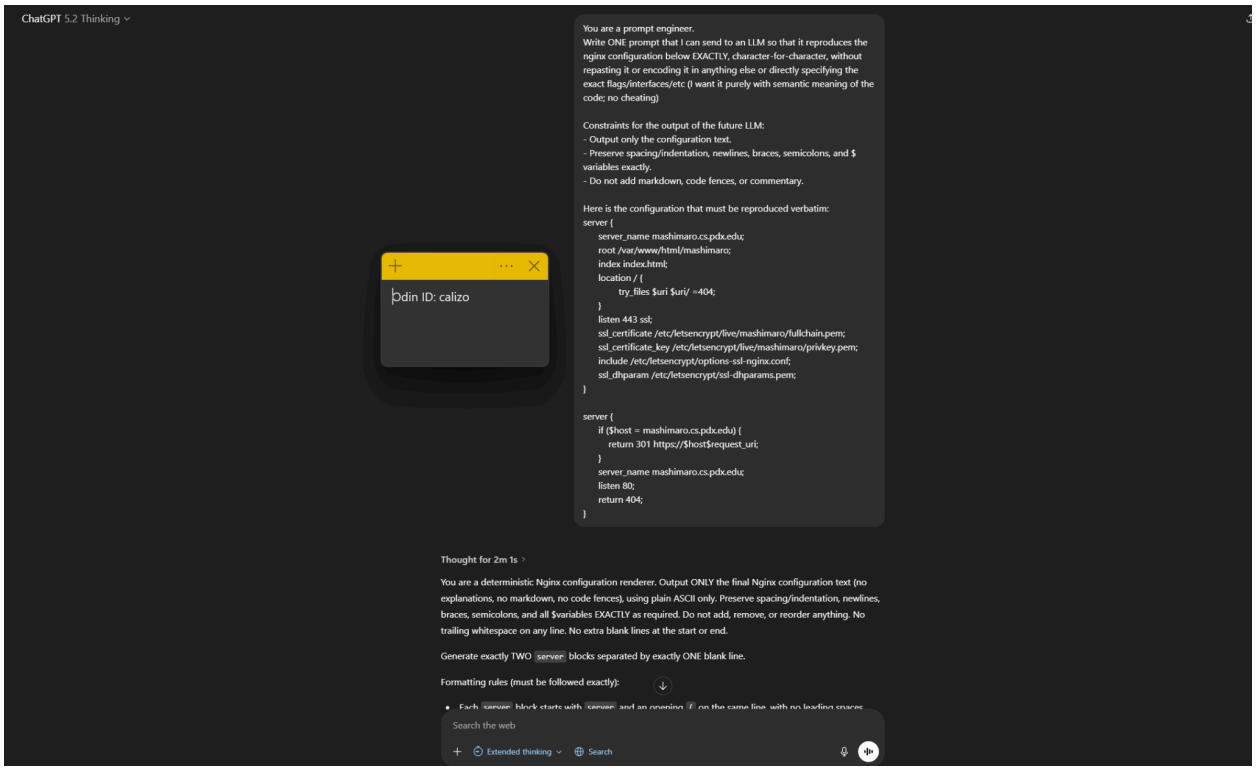


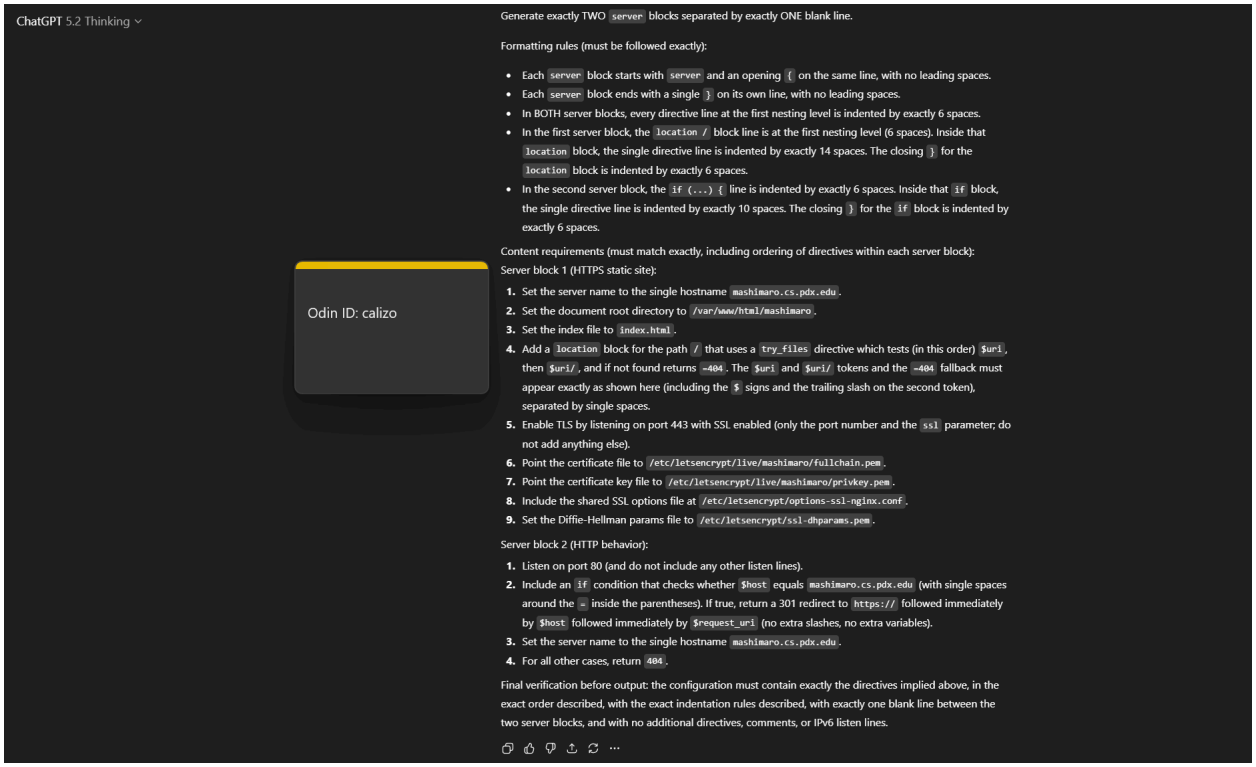
Step 5. nginx configuration

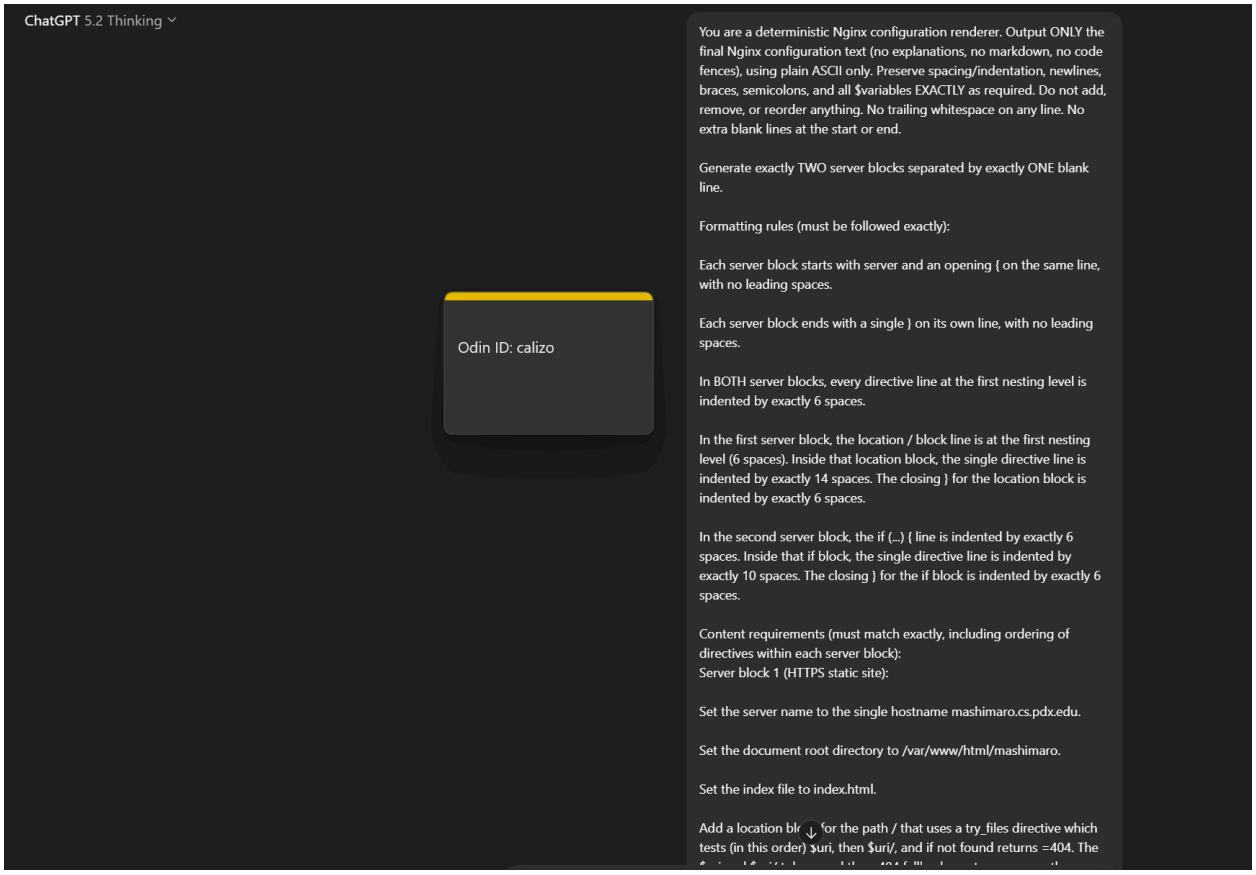
1. Take a screenshot of the result that includes your OdinID

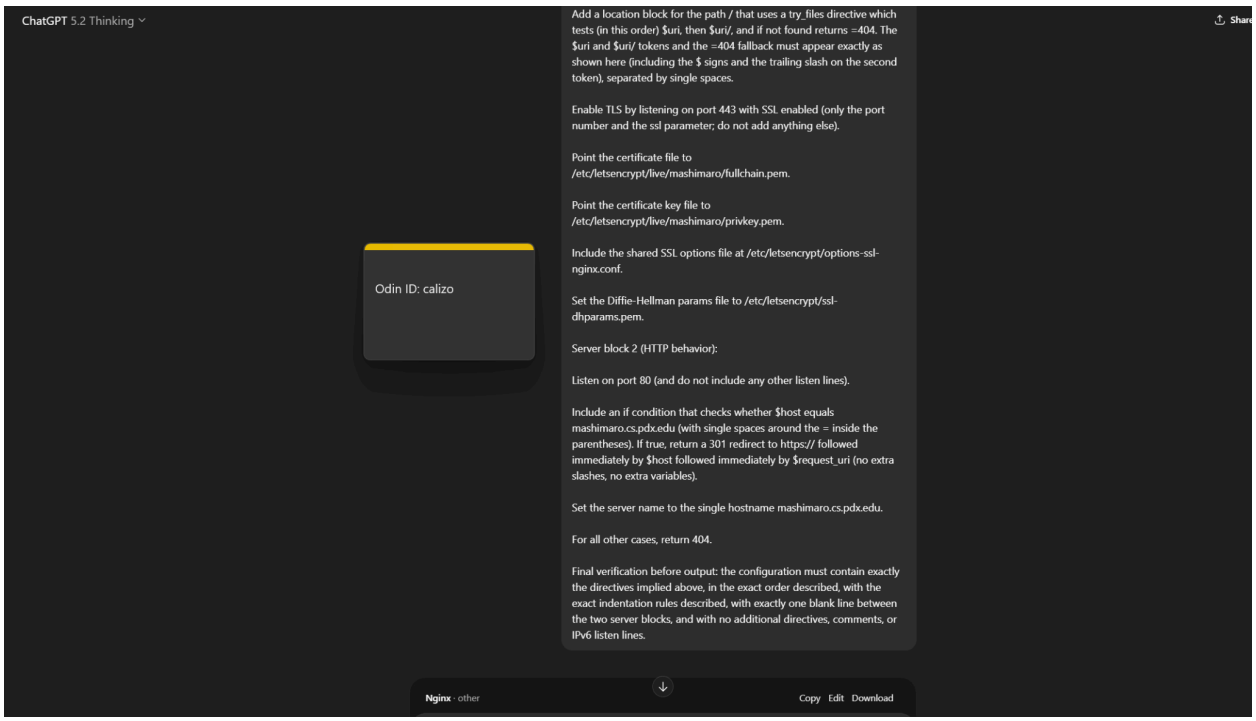


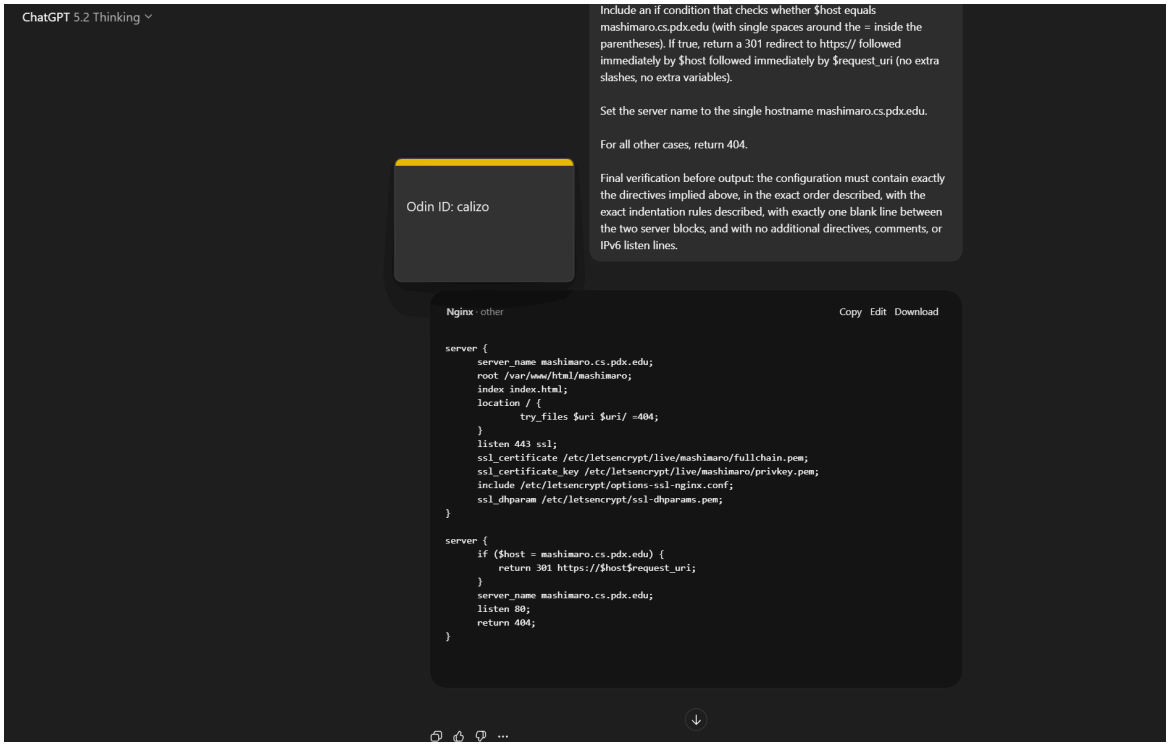
2. Take a screenshot of the prompt and result that includes your OdinID





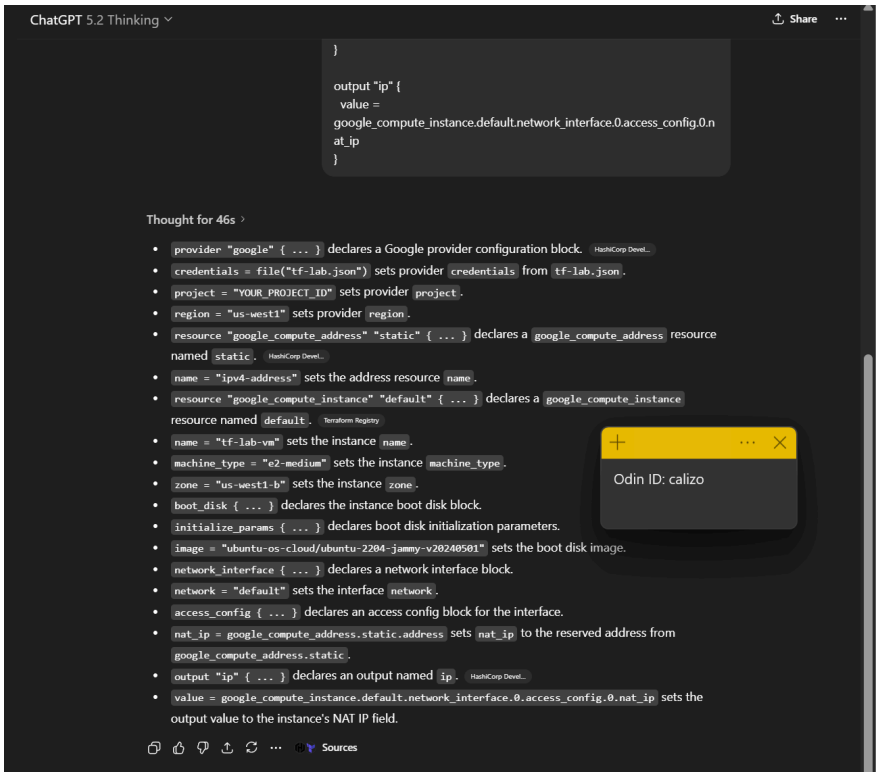

ChatGPT 5.2 Thinking
Odin ID: calizo
You are a deterministic Nginx configuration renderer. Output ONLY the final Nginx configuration text (no explanations, no markdown, no code fences), using plain ASCII only. Preserve spacing/indentation, newlines, braces, semicolons, and all $variables EXACTLY as required. Do not add, remove, or reorder anything. No trailing whitespace on any line. No extra blank lines at the start or end.
Generate exactly TWO server blocks separated by exactly ONE blank line.
Formatting rules (must be followed exactly):
Each server block starts with server and an opening { on the same line, with no leading spaces.
Each server block ends with a single } on its own line, with no leading spaces.
In BOTH server blocks, every directive line at the first nesting level is indented by exactly 6 spaces.
In the first server block, the location / block line is at the first nesting level (6 spaces). Inside that location block, the single directive line is indented by exactly 14 spaces. The closing } for the location block is indented by exactly 6 spaces.
In the second server block, the if (...) { line is indented by exactly 6 spaces. Inside that if block, the single directive line is indented by exactly 10 spaces. The closing } for the if block is indented by exactly 6 spaces.
Content requirements (must match exactly, including ordering of directives within each server block):
Server block 1 (HTTPS static site):
Set the server name to the single hostname mashimaro.cs.pdx.edu.
Set the document root directory to /var/www/html/mashimaro.
Set the index file to index.html.
Add a location bl for the path / that uses a try_files directive which tests (in this order) $uri, then $uri/, and if not found returns =404. The


ChatGPT 5.2 Thinking
Odin ID: calizo
Add a location block for the path / that uses a try_files directive which tests (in this order) $uri, then $uri/, and if not found returns =404. The $uri and $uri/ tokens and the =404 fallback must appear exactly as shown here (including the $ signs and the trailing slash on the second token), separated by single spaces.
Enable TLS by listening on port 443 with SSL enabled (only the port number and the ssl parameter; do not add anything else).
Point the certificate file to /etc/letsencrypt/live/mashimaro/fullchain.pem.
Point the certificate key file to /etc/letsencrypt/live/mashimaro/privkey.pem.
Include the shared SSL options file at /etc/letsencrypt/options-ssl-nginx.conf.
Set the Diffie-Hellman params file to /etc/letsencrypt/ssl-dhparams.pem.
Server block 2 (HTTP behavior):
Listen on port 80 (and do not include any other listen lines).
Include an if condition that checks whether $host equals mashimaro.cs.pdx.edu (with single spaces around the = inside the parentheses). If true, return a 301 redirect to https:// followed immediately by $host followed immediately by $request_uri (no extra slashes, no extra variables).
Set the server name to the single hostname mashimaro.cs.pdx.edu.
For all other cases, return 404.
Final verification before output: the configuration must contain exactly the directives implied above, in the exact order described, with the exact indentation rules described, with exactly one blank line between the two server blocks, and with no additional directives, comments, or IPv6 listen lines.
Nginx · other
Copy Edit Download

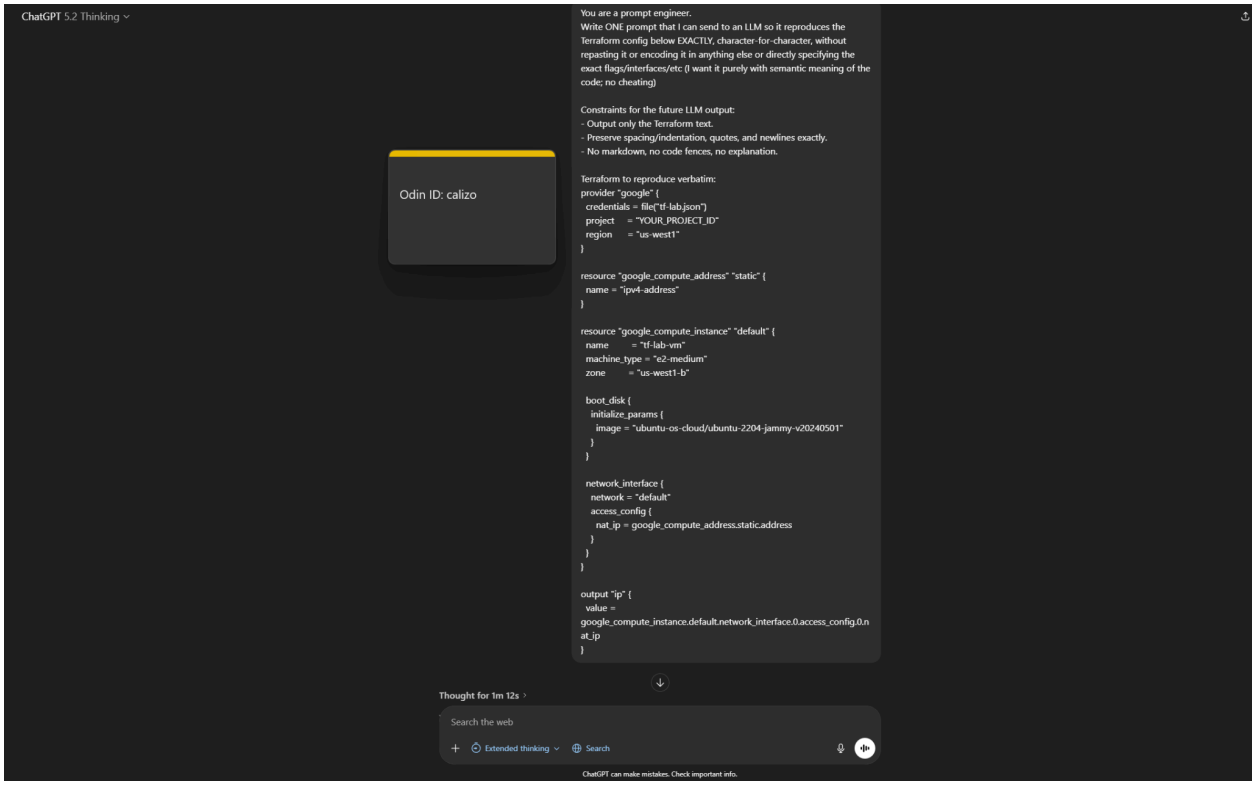


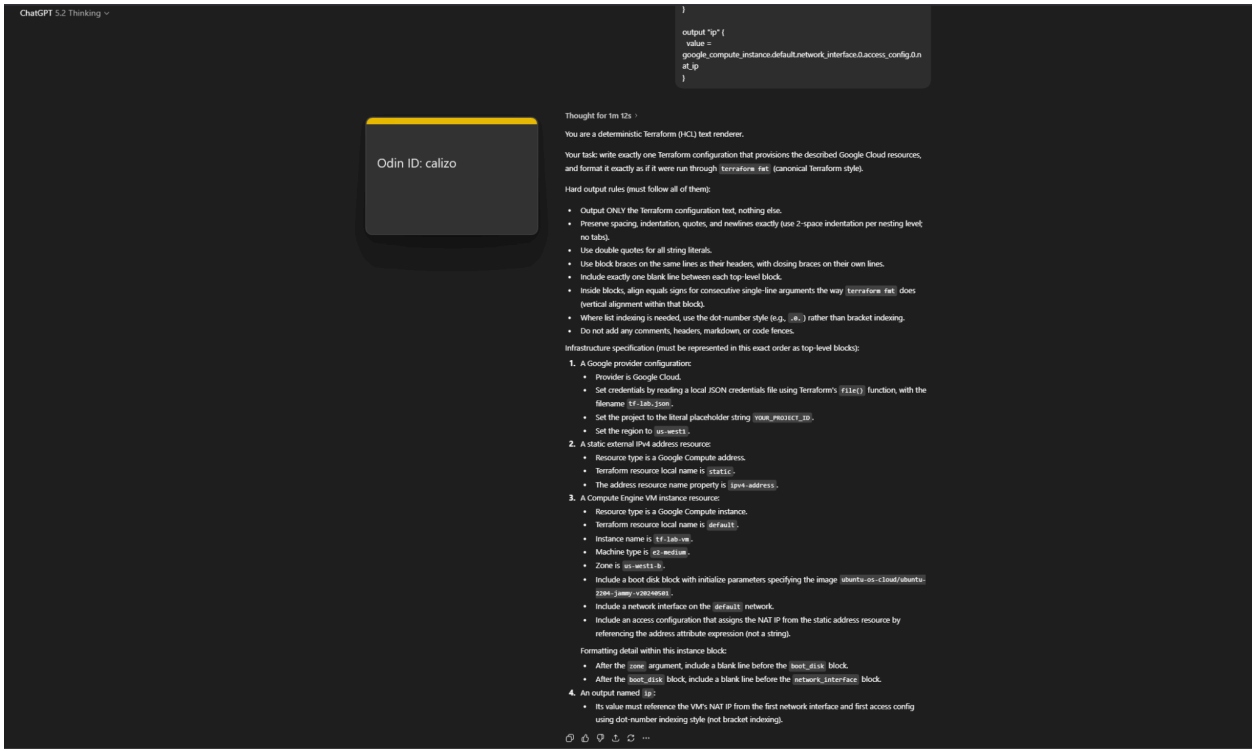
Step 6. Terraform configuration

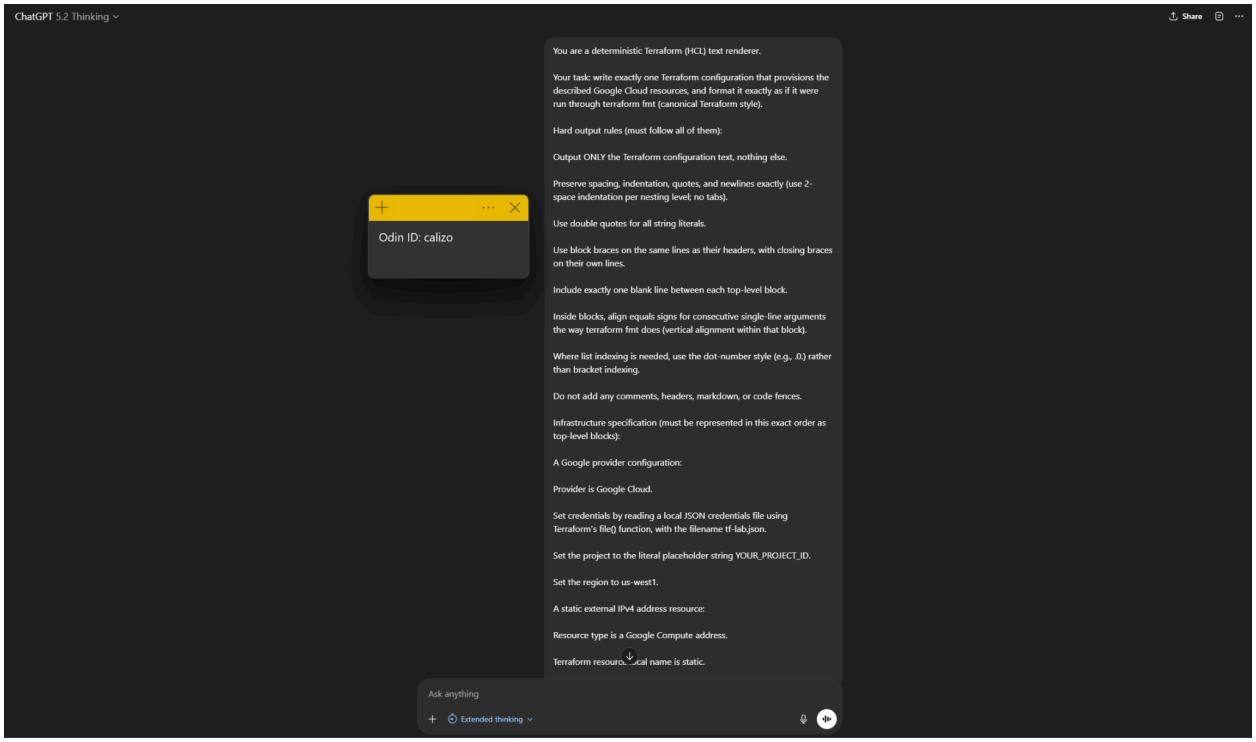
1. Take a screenshot of the result that includes your OdinID

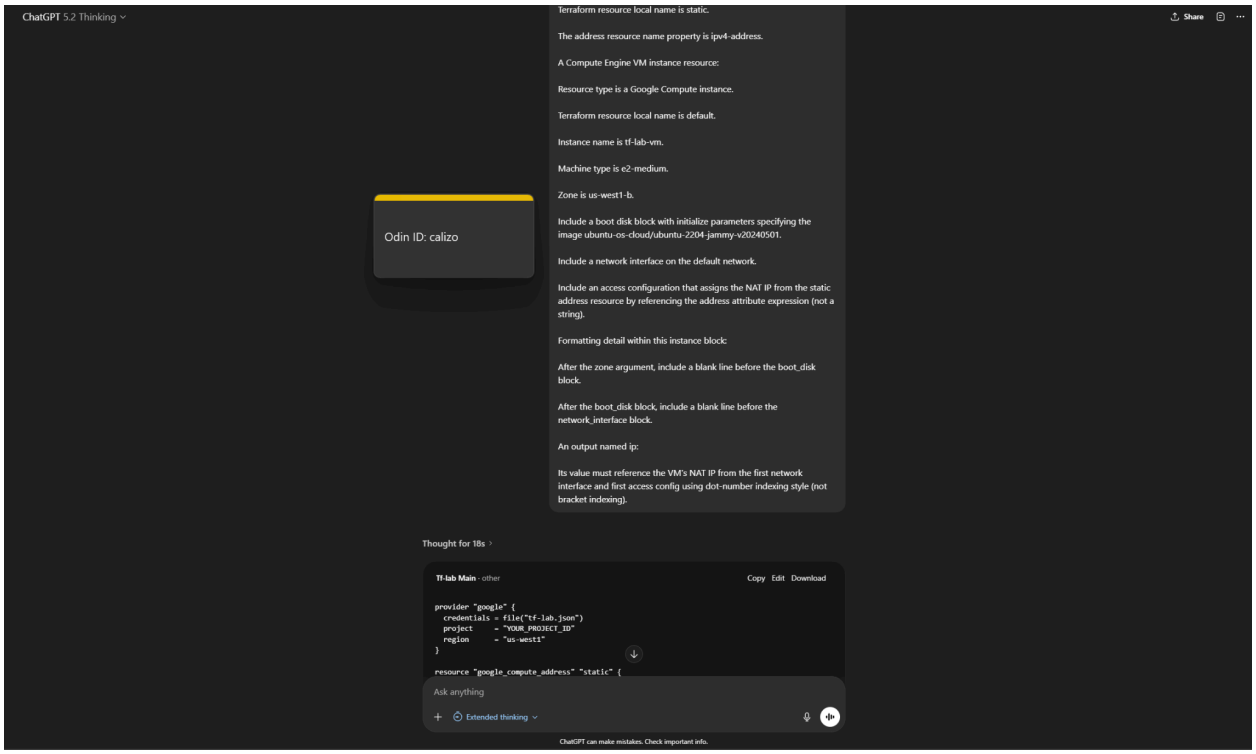


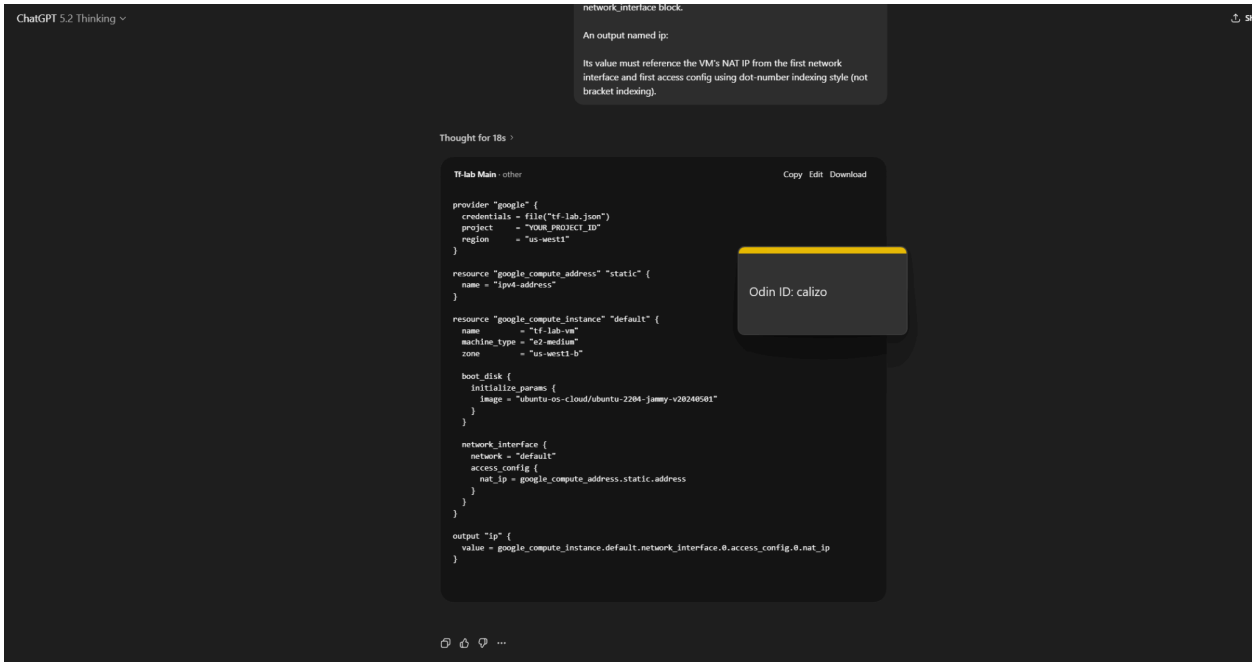
2. Take a screenshot of the prompt and result that includes your OdinID





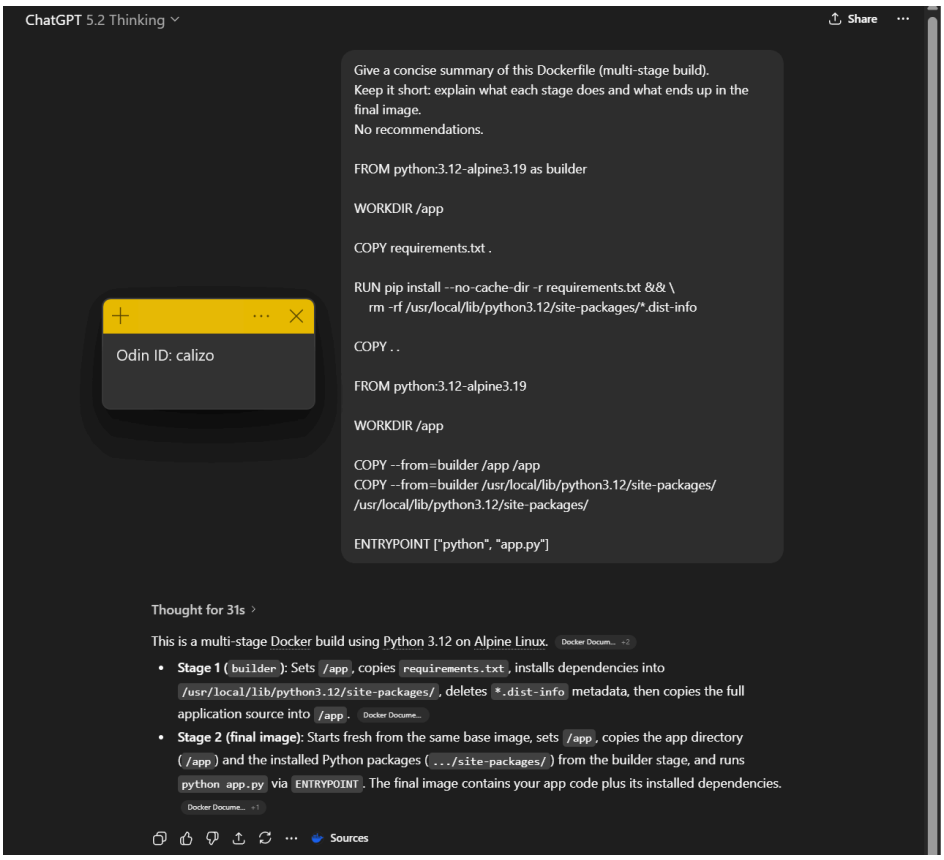

ChatGPT 5.2 Thinking
Share
Odin ID: calizo
You are a deterministic Terraform (HCL) text renderer.
Your task: write exactly one Terraform configuration that provisions the described Google Cloud resources, and format it exactly as if it were run through terraform fmt (canonical Terraform style).
Hard output rules (must follow all of them):
Output ONLY the Terraform configuration text, nothing else.
Preserve spacing, indentation, quotes, and newlines exactly (use 2-space indentation per nesting level; no tabs).
Use double quotes for all string literals.
Use block braces on the same lines as their headers, with closing braces on their own lines.
Include exactly one blank line between each top-level block.
Inside blocks, align equals signs for consecutive single-line arguments the way terraform fmt does (vertical alignment within that block).
Where list indexing is needed, use the dot-number style (e.g., .0.) rather than bracket indexing.
Do not add any comments, headers, markdown, or code fences.
Infrastructure specification (must be represented in this exact order as top-level blocks):
A Google provider configuration:
Provider is Google Cloud.
Set credentials by reading a local JSON credentials file using Terraform's file() function, with the filename tf-lab.json.
Set the project to the literal placeholder string YOUR_PROJECT_ID.
Set the region to us-west1.
A static external IPv4 address resource:
Resource type is a Google Compute address.
Terraform resource local name is static.
Ask anything
Extended thinking


ChatGPT 5.2 Thinking
Share
Odin ID: calizo
Terraform resource local name is static.
The address resource name property is ipv4-address.
A Compute Engine VM instance resource:
Resource type is a Google Compute instance.
Terraform resource local name is default.
Instance name is tf-lab-vm.
Machine type is e2-medium.
Zone is us-west1-b.
Include a boot disk block with initialize parameters specifying the image ubuntu-os-cloud/ubuntu-2204-jammy-v20240501.
Include a network interface on the default network.
Include an access configuration that assigns the NAT IP from the static address resource by referencing the address attribute expression (not a string).
Formatting detail within this instance block:
After the zone argument, include a blank line before the boot_disk block.
After the boot_disk block, include a blank line before the network_interface block.
An output named ip:
Its value must reference the VM's NAT IP from the first network interface and first access config using dot-number indexing style (not bracket indexing).
Thought for 18s
Tf-lab Main · other
Copy Edit Download
provider "google" {
credentials = file("tf-lab.json")
project = "YOUR_PROJECT_ID"
region = "us-west1"
}
resource "google_compute_address" "static" {
Ask anything
Extended thinking
ChatGPT can make mistakes. Check important info.

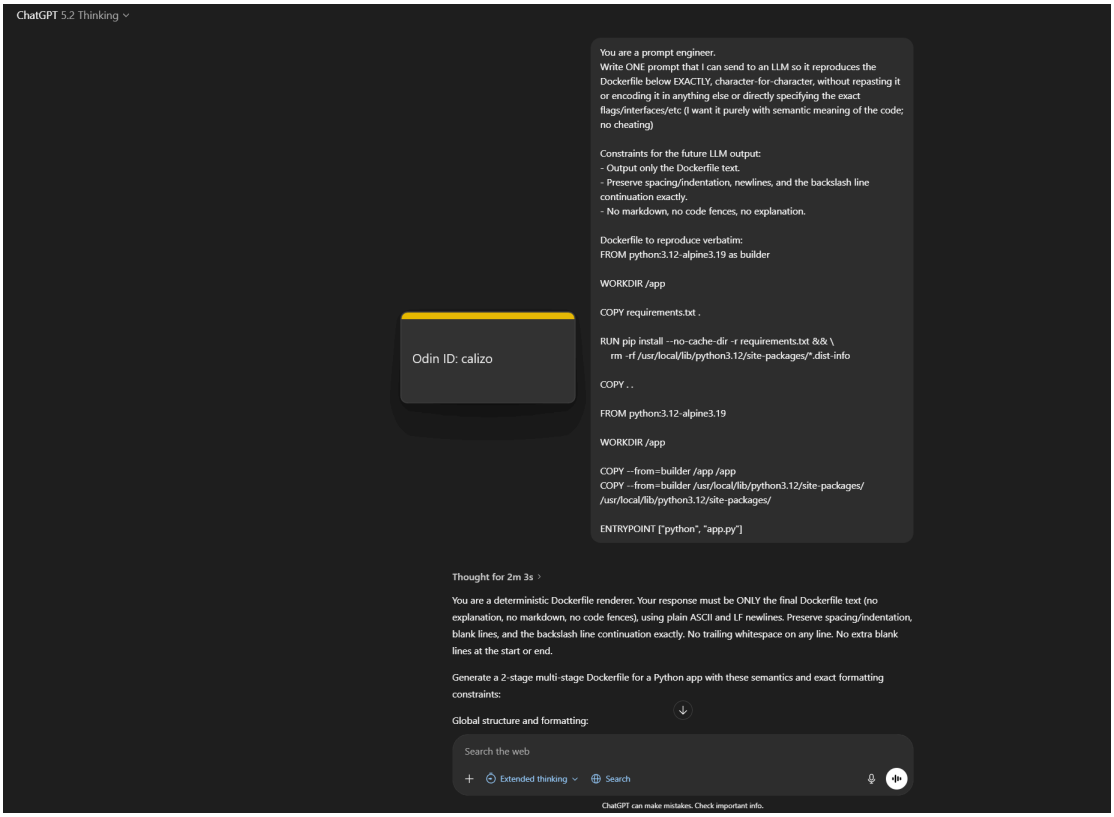


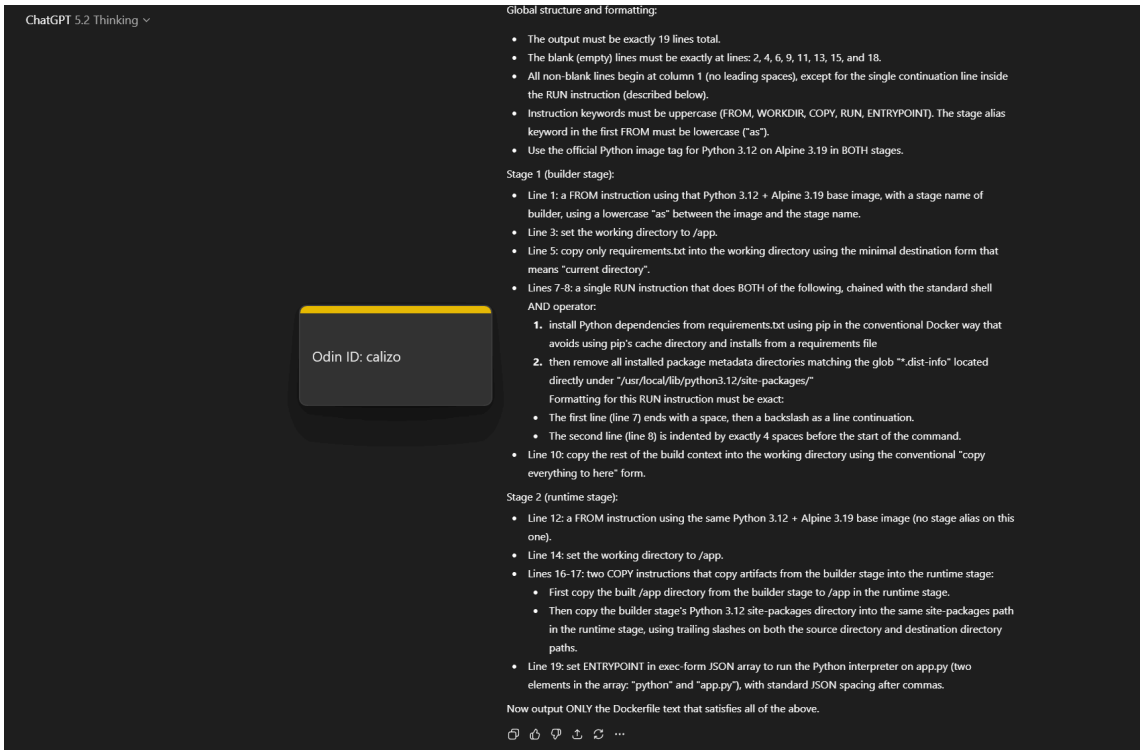
Step 7. Docker configuration

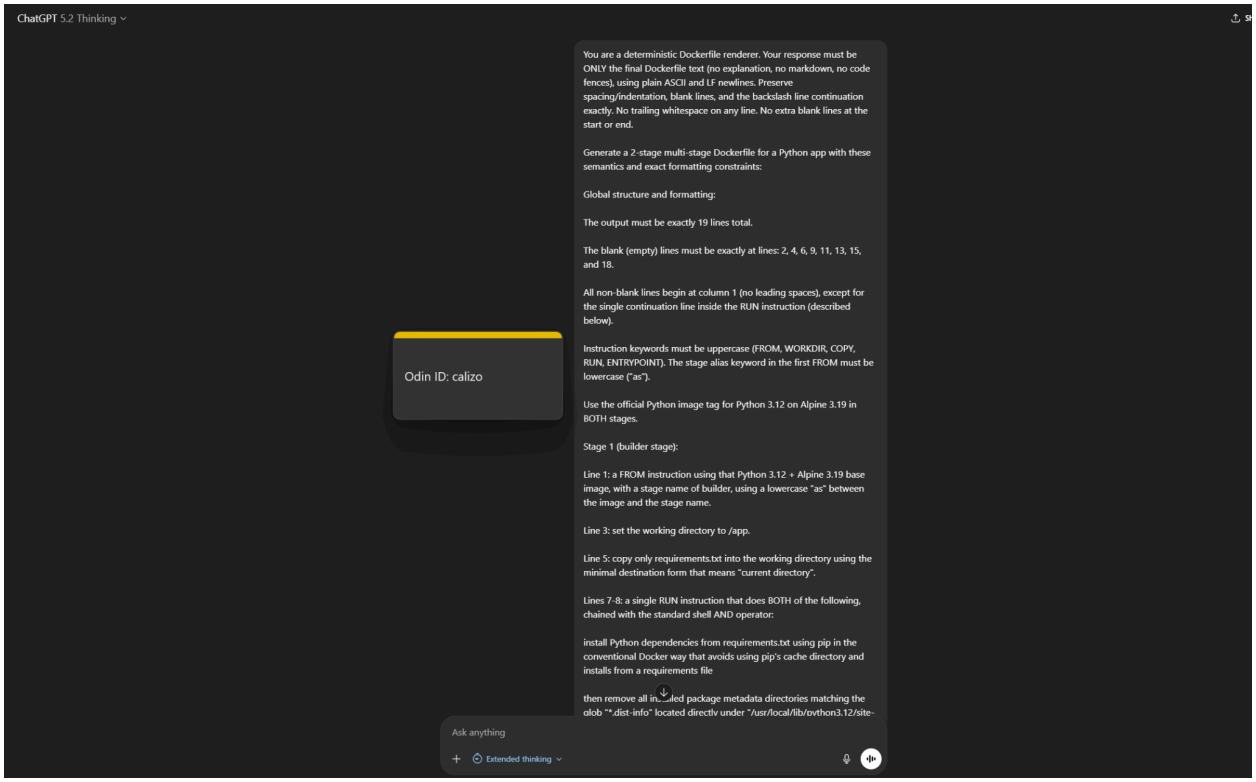
1. Take a screenshot of the result that includes your OdinID

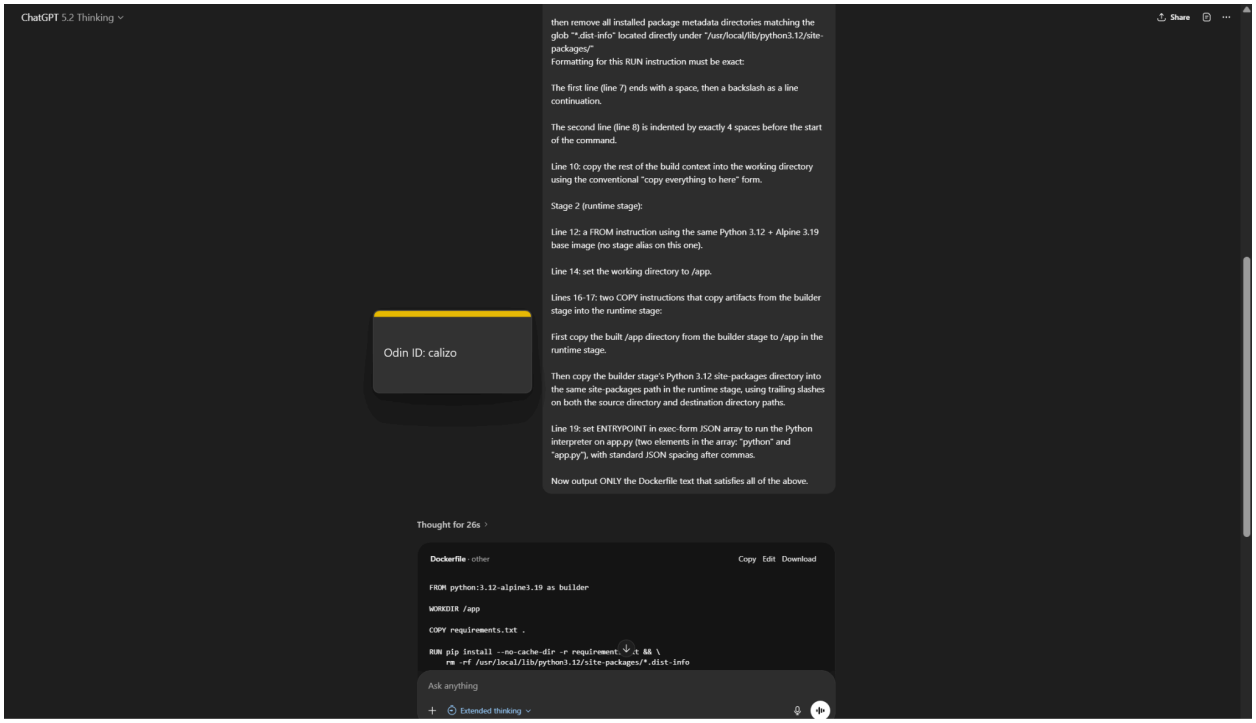


2. Take a screenshot of the prompt and result that includes your OdinID






ChatGPT 5.2 Thinking
You are a deterministic Dockerfile renderer. Your response must be ONLY the final Dockerfile text (no explanation, no markdown, no code fences), using plain ASCII and LF newlines. Preserve spacing/indentation, blank lines, and the backslash line continuation exactly. No trailing whitespace on any line. No extra blank lines at the start or end.
Generate a 2-stage multi-stage Dockerfile for a Python app with these semantics and exact formatting constraints:
Global structure and formatting:
The output must be exactly 19 lines total.
The blank (empty) lines must be exactly at lines: 2, 4, 6, 9, 11, 13, 15, and 18.
All non-blank lines begin at column 1 (no leading spaces), except for the single continuation line inside the RUN instruction (described below).
Instruction keywords must be uppercase (FROM, WORKDIR, COPY, RUN, ENTRYPOINT). The stage alias keyword in the first FROM must be lowercase ("as").
Use the official Python image tag for Python 3.12 on Alpine 3.19 in BOTH stages.
Stage 1 (builder stage):
Line 1: a FROM instruction using that Python 3.12 + Alpine 3.19 base image, with a stage name of builder, using a lowercase "as" between the image and the stage name.
Line 3: set the working directory to /app.
Line 5: copy only requirements.txt into the working directory using the minimal destination form that means "current directory".
Lines 7-8: a single RUN instruction that does BOTH of the following, chained with the standard shell AND operator:
install Python dependencies from requirements.txt using pip in the conventional Docker way that avoids using pip's cache directory and installs from a requirements file
then remove all in
olob "*.dist-info" located directly under "/usr/local/lib/python3.12/site-
Odin ID: calizo
Ask anything
Extended thinking


ChatGPT 5.2 Thinking
Share
then remove all installed package metadata directories matching the glob "*.dist-info" located directly under "/usr/local/lib/python3.12/site-packages/"
Formatting for this RUN instruction must be exact:
The first line (line 7) ends with a space, then a backslash as a line continuation.
The second line (line 8) is indented by exactly 4 spaces before the start of the command.
Line 10: copy the rest of the build context into the working directory using the conventional "copy everything to here" form.
Stage 2 (runtime stage):
Line 12: a FROM instruction using the same Python 3.12 + Alpine 3.19 base image (no stage alias on this one).
Line 14: set the working directory to /app.
Lines 16-17: two COPY instructions that copy artifacts from the builder stage into the runtime stage:
First copy the built /app directory from the builder stage to /app in the runtime stage.
Then copy the builder stage's Python 3.12 site-packages directory into the same site-packages path in the runtime stage, using trailing slashes on both the source directory and destination directory paths.
Line 19: set ENTRYPOINT in exec-form JSON array to run the Python interpreter on app.py (two elements in the array: "python" and "app.py"), with standard JSON spacing after commas.
Now output ONLY the Dockerfile text that satisfies all of the above.
Odin ID: calizo
Thought for 26s
Dockerfile · other
Copy Edit Download
FROM python:3.12-alpine3.19 as builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirement t && \
rm -rf /usr/local/lib/python3.12/site-packages/*.dist-info
Ask anything
Extended thinking

Now output ONLY the Dockerfile text that satisfies all of the above.

Thought for 26s ›

Dockerfile · other Copy Edit Download

```
FROM python:3.12-alpine3.19 as builder

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt && \
    rm -rf /usr/local/lib/python3.12/site-packages/*.dist-info

COPY . .

FROM python:3.12-alpine3.19

WORKDIR /app

COPY --from=builder /app /app
COPY --from=builder /usr/local/lib/python3.12/site-packages/ /usr/local/lib/python3.12/site-pacl

ENTRYPOINT ["python", "app.py"]
```

Odin ID: calizo

Thought for 26s ›

Dockerfile · other Copy Edit Download

```
ython:3.12-alpine3.19 as builder

R /app

equirements.txt .

p install --no-cache-dir -r requirements.txt && \
 -rf /usr/local/lib/python3.12/site-packages/*.dist-info

 .

ython:3.12-alpine3.19

R /app

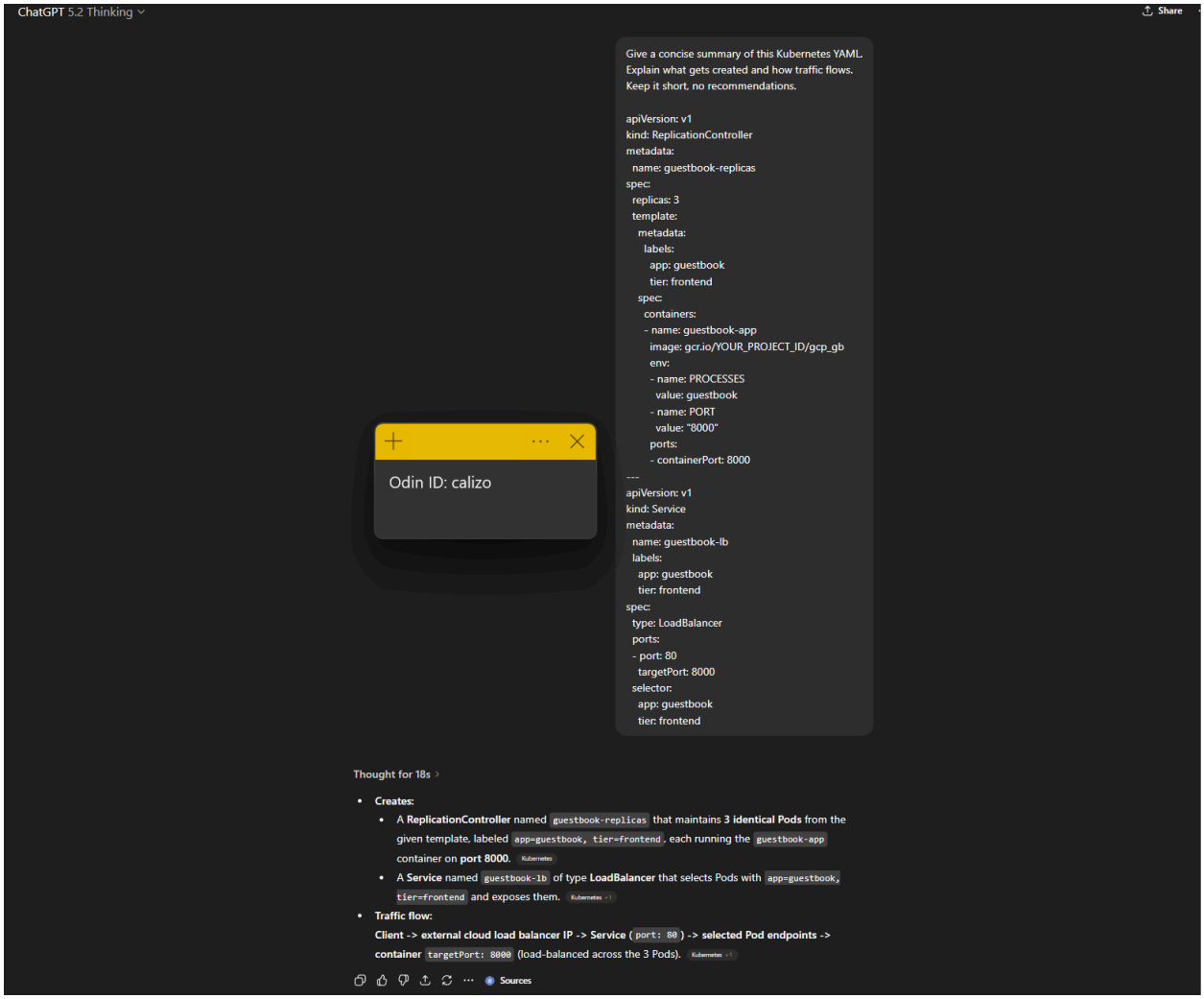
-from=builder /app /app
-from=builder /usr/local/lib/python3.12/site-packages/ /usr/local/lib/python3.12/site-packages/

OINT ["python", "app.py"]
```

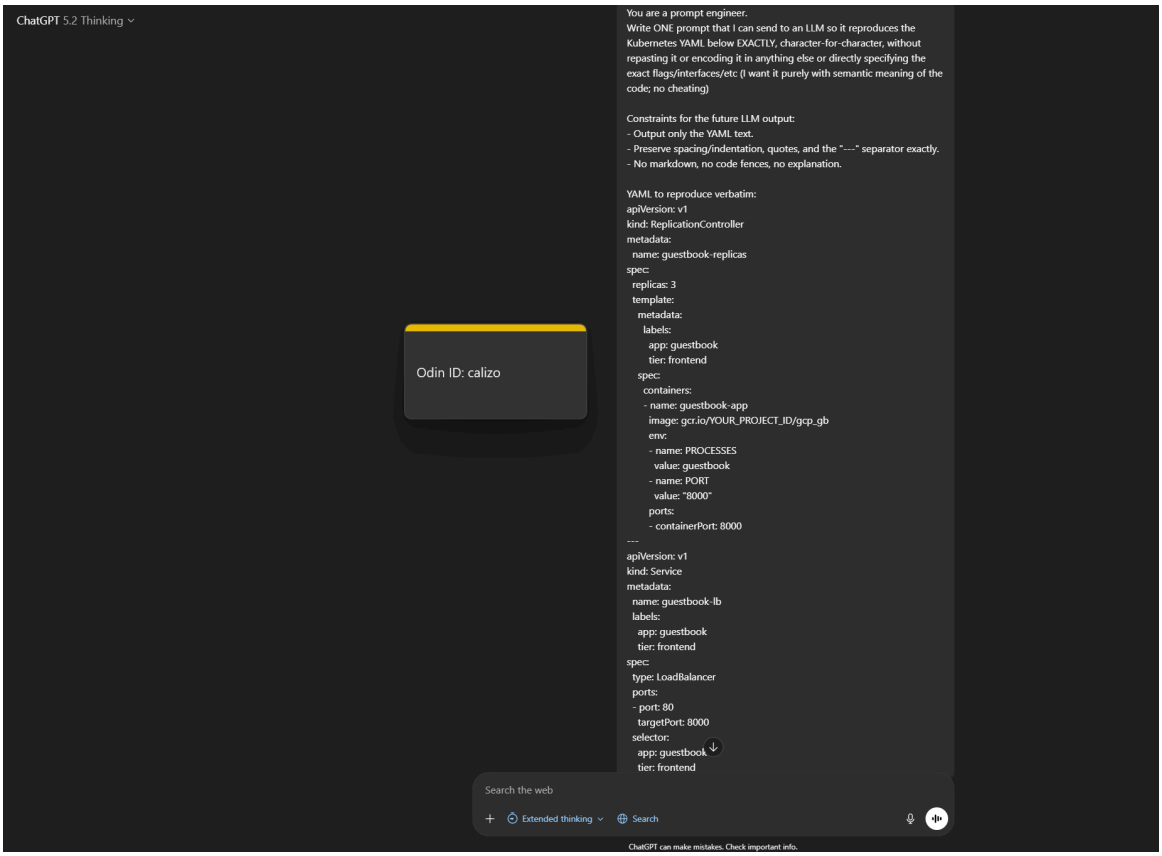
Odin ID: calizo

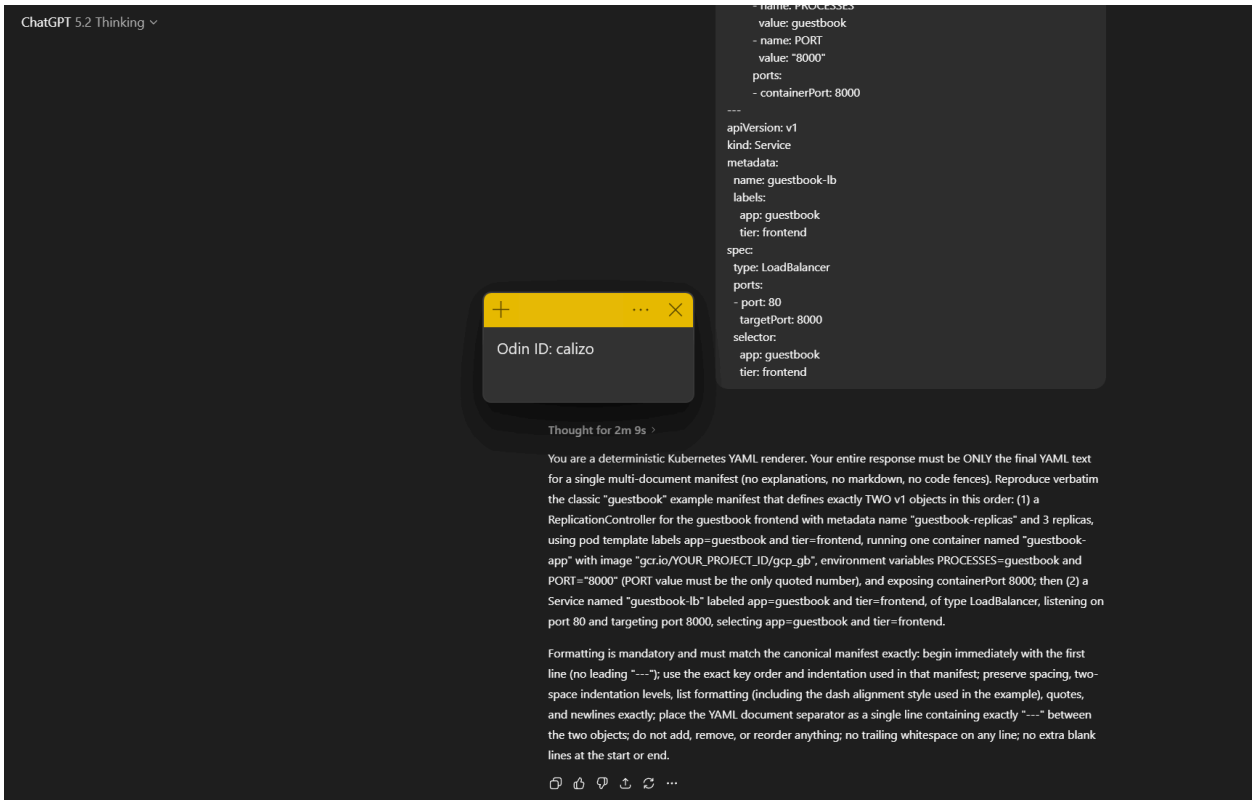
Step 8. Kubernetes configuration

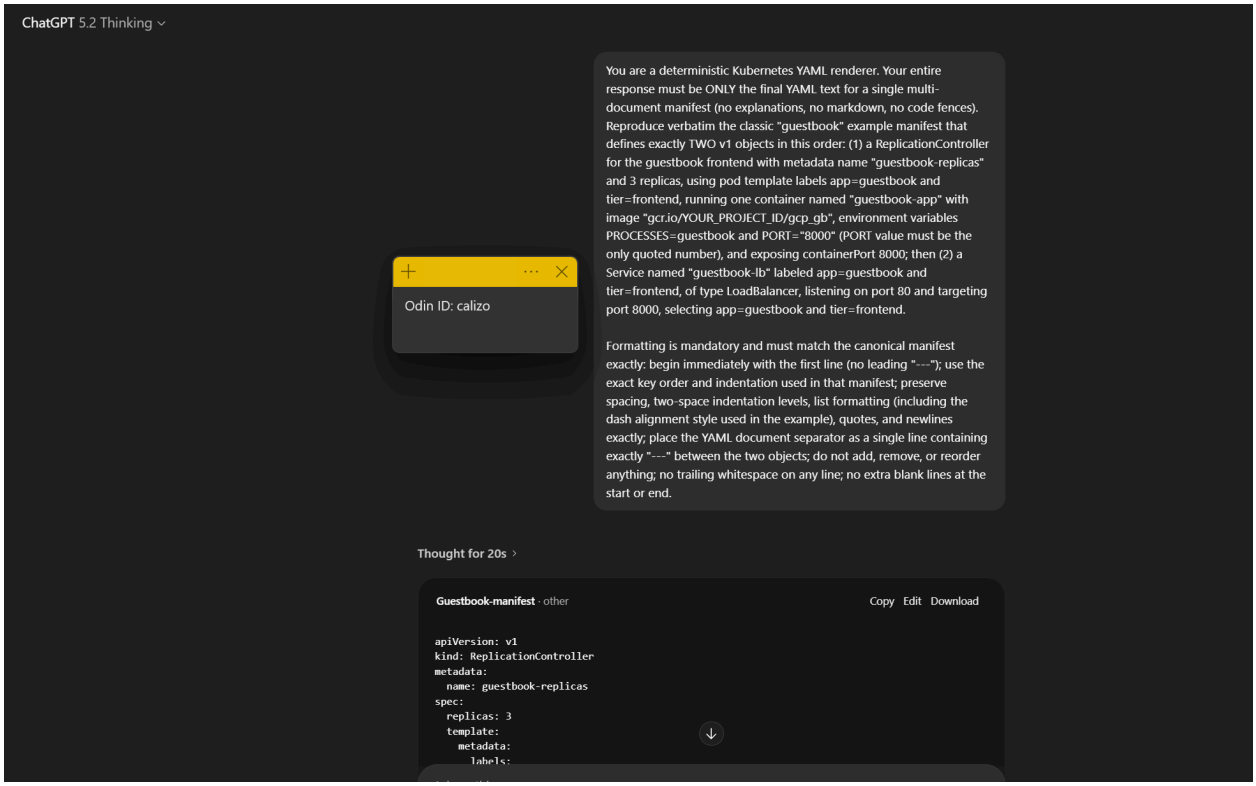
1. Take a screenshot of the result that includes your OdinID

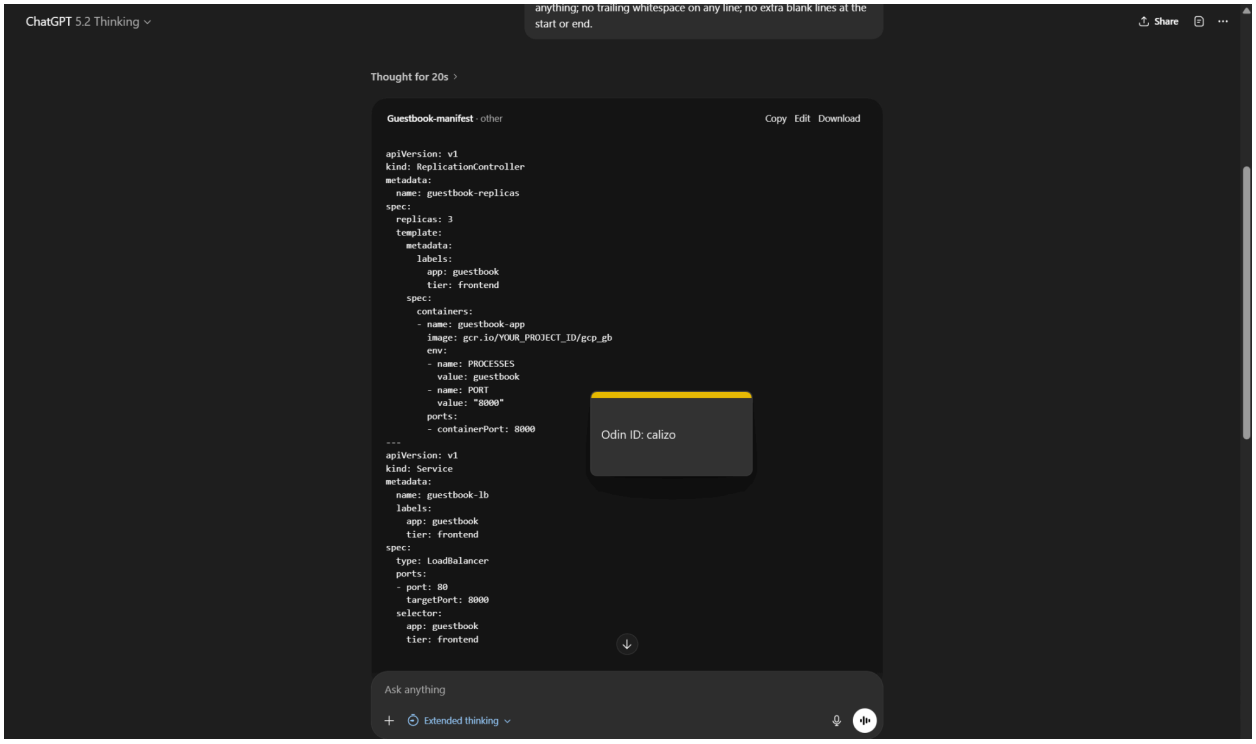


2. Take a screenshot of the prompt and result that includes your OdinID






ChatGPT 5.2 Thinking
You are a deterministic Kubernetes YAML renderer. Your entire response must be ONLY the final YAML text for a single multi-document manifest (no explanations, no markdown, no code fences). Reproduce verbatim the classic "guestbook" example manifest that defines exactly TWO v1 objects in this order: (1) a ReplicationController for the guestbook frontend with metadata name "guestbook-replicas" and 3 replicas, using pod template labels app=guestbook and tier=frontend, running one container named "guestbook-app" with image "gcr.io/YOUR_PROJECT_ID/gcp_gb", environment variables PROCESSES=guestbook and PORT="8000" (PORT value must be the only quoted number), and exposing containerPort 8000; then (2) a Service named "guestbook-lb" labeled app=guestbook and tier=frontend, of type LoadBalancer, listening on port 80 and targeting port 8000, selecting app=guestbook and tier=frontend.
Formatting is mandatory and must match the canonical manifest exactly: begin immediately with the first line (no leading "---"); use the exact key order and indentation used in that manifest; preserve spacing, two-space indentation levels, list formatting (including the dash alignment style used in the example), quotes, and newlines exactly; place the YAML document separator as a single line containing exactly "---" between the two objects; do not add, remove, or reorder anything; no trailing whitespace on any line; no extra blank lines at the start or end.
Odin ID: calizo
Thought for 20s
Guestbook-manifest · other
Copy Edit Download
apiVersion: v1
kind: ReplicationController
metadata:
name: guestbook-replicas
spec:
replicas: 3
template:
metadata:
labels:


ChatGPT 5.2 Thinking
anything; no trailing whitespace on any line; no extra blank lines at the start or end.
Share
Thought for 20s
Guestbook-manifest · other
Copy Edit Download
apiVersion: v1
kind: ReplicationController
metadata:
name: guestbook-replicas
spec:
replicas: 3
template:
metadata:
labels:
app: guestbook
tier: frontend
spec:
containers:
- name: guestbook-app
image: gcr.io/YOUR_PROJECT_ID/gcp_gb
env:
- name: PROCESSES
value: guestbook
- name: PORT
value: "8000"
ports:
- containerPort: 8000
---
apiVersion: v1
kind: Service
metadata:
name: guestbook-lb
labels:
app: guestbook
tier: frontend
spec:
type: LoadBalancer
ports:
- port: 80
targetPort: 8000
selector:
app: guestbook
tier: frontend
Odin ID: calizo
Ask anything
Extended thinking

Lab 7.2 Command Agents

Step 1. Linux processes

1. Include a table of successful commands generated for each task for your notebook

Task	Agent Prompt	Tool Call Command(s) Ran
Open network connections	Using sudo and lsof, list process names that currently have open network connections.	sudo lsof -i -n -P
sshd exe path	Find the sshd PID and show the executable path from /proc.	ps -C sshd -o pid= [could use 'ps aux'] sudo ls -l /proc/1174/exe
Deleted binaries	Using sudo and lsof, show processes whose executable or mapped file is deleted.	sudo lsof +L1 -a -d mem
Python cwd	Find a python PID and show its current working directory from /proc.	ps -C python -o pid= [could use 'ps aux'] sudo ls -l /proc/1446/cwd
Python environ	Find python PIDs, then show the environment of one python process from /proc/<PID>/environ.	ps -C python -o pid= [could use 'ps aux'] sudo ls -l /proc/1271/environ
cwd is /tmp	List processes whose /proc/<PID>/cwd points to /tmp. Use sudo if needed.	sudo lsof -a -d cwd /tmp
Scheduled tasks	Show scheduled tasks on this system (cron locations and system crontabs).	sudo ls -la /etc/crontab sudo ls -la /etc/cron.d/ sudo ls -la /etc/cron.hourly/ sudo ls -la

		/etc/cron.daily/ sudo ls -la /etc/cron.weekly/ sudo ls -la /etc/cron.monthly/ sudo ls -la /var/spool/cron/crontabs /

Step 2. Linux files and regular expressions (grep)

1. Include a table of successful commands generated for each task for your notebook

Task	Agent Prompt	Tool Call Command(s) Ran
Start with a	Count passwords that start with the letter a.	egrep -c '^a' /usr/share/wordlists/rockyou.txt
Start with 1	Count passwords that start with the number 1.	egrep -c '^1' /usr/share/wordlists/rockyou.txt
Only letters	Count passwords made only of letters A-Z (no digits, no symbols).	egrep -c '^[A-Za-z]+$' /usr/share/wordlists/rockyou.txt
Exactly 6 digits	Count passwords that are exactly 6 digits long.	egrep -c '^[0-9]{6}$' /usr/share/wordlists/rockyou.txt
Start abcde end 9	List passwords that start with abcde and end with 9.	egrep '^abcde.*9$' /usr/share/wordlists/rockyou.txt

Step 3. Linux file searching (find)

1. Include a table of successful commands generated for each task for your notebook

Task	Agent Prompt	Tool Call Command(s) Ran
setuid/setgid in /bin	Using sudo and find, list all files in /bin with setuid or setgid bits set	sudo find /bin -maxdepth 1 -type f -perm /6000 -print
ELF in /bin	Find ELF executables in /bin	sudo find /bin -maxdepth 1 -type f -executable -print
modified last day /etc	Using sudo and find, list files in /etc modified in the last 24 hours	sudo find /etc -type f -mtime 0 -print
created last day /etc	Using sudo and find, list files in /etc created in the last 24 hours	sudo find /etc -type f -newerBt "1 day ago" -print

Step 4. Linux firewall

1. Include a table of successful commands generated for each task for your notebook

Rule Request	Agent Prompt	iptables Command(s) Ran
allow ssh from 131.252.220.0/24	Using sudo and find, list all files in /bin with setuid or setgid bits set	sudo iptables -A INPUT -p tcp -s 131.252.220.0/24 --dport 22 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT sudo iptables -A OUTPUT -p tcp -d 131.252.220.0/24 --sport 22 -m conntrack --ctstate ESTABLISHED -j

		ACCEPT
allow 80/443 from 131.252.0.0/16	Find ELF executables in /bin	sudo iptables -A INPUT -p tcp -s 131.252.0.0/16 -m multiport --dports 80,443 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT sudo iptables -A OUTPUT -p tcp -d 131.252.0.0/16 -m multiport --sports 80,443 -m conntrack --ctstate ESTABLISHED -j ACCEPT
allow outbound mysql to 10.0.0.10	Using sudo and find, list files in /etc modified in the last 24 hours	sudo iptables -A OUTPUT -p tcp -d 10.0.0.10 --dport 3306 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT sudo iptables -A INPUT -p tcp -s 10.0.0.10 --sport 3306 -m conntrack --ctstate ESTABLISHED -j ACCEPT
drop traffic from 1.1.1.1	Using sudo and find, list files in /etc created in the last 24 hours	sudo iptables -A INPUT -s 1.1.1.1 -j DROP

Step 5. Linux audit logs

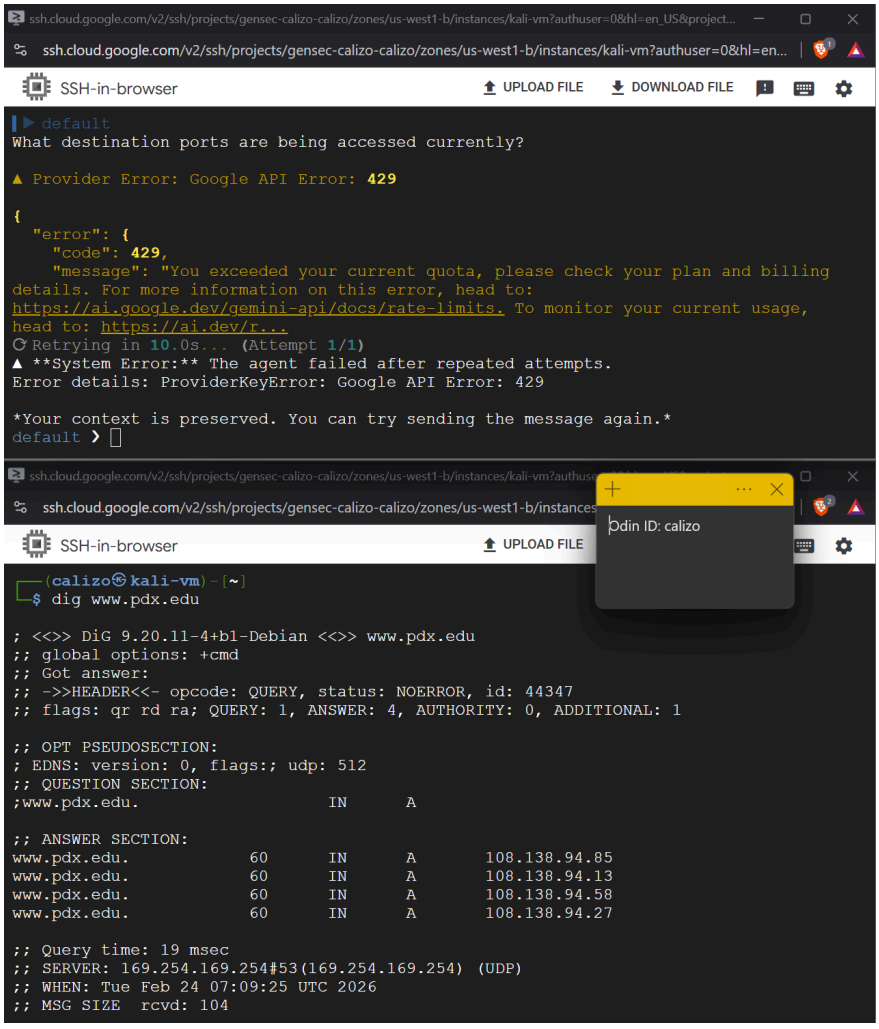
1. Include a table of successful commands generated for each task for your notebook

Query	Agent Prompt	Tool call Command(s) Ran
current users	Show all currently logged in users.	sudo who

last 5 successful	Show the last 5 successful logins.	sudo last | egrep -v '^(reboot|shutdown|runlevel|wtmp begins)' | head -n 5
last 5 failed and IP	Show the last 5 failed logins and include the source IP address.	sudo lastb -i -n 5

Step 6. tcpdump

1. Take a screenshot showing the results for each that includes your OdinId



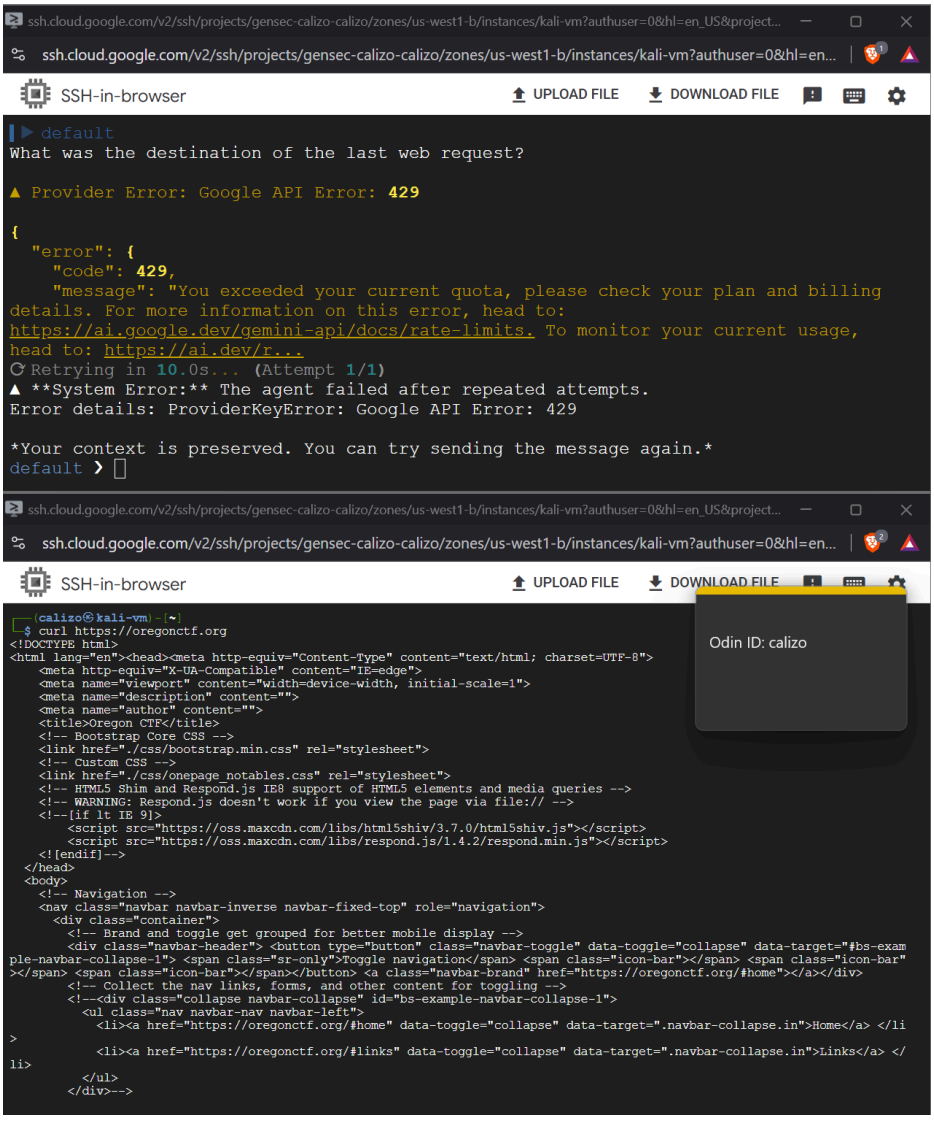
Expected Output: A traffic loopback to 127.0.0.53:53 or Query going directly to some resolver (router/campus/8.8.8.8)

- UDP/53 (but if UDP is truncated, it will TCP fallback to port 53; TCP/53)

- IP 35.199.165.186.<ephemeral_src_port> > 169.254.169.254.53: 44347+ A? www.pdx.edu. (29)

TCPdump:

- IP 169.254.169.254.53 > 35.199.165.186.<ephemeral_src_port>: 44347* 4/0/1 A 108.138.94.85, A 108.138.94.13, A 108.138.94.58, A 108.138.94.27 (104)
    - one UDP packet to destination port 53 carrying the query, then one UDP packet back from port 53 carrying the answer



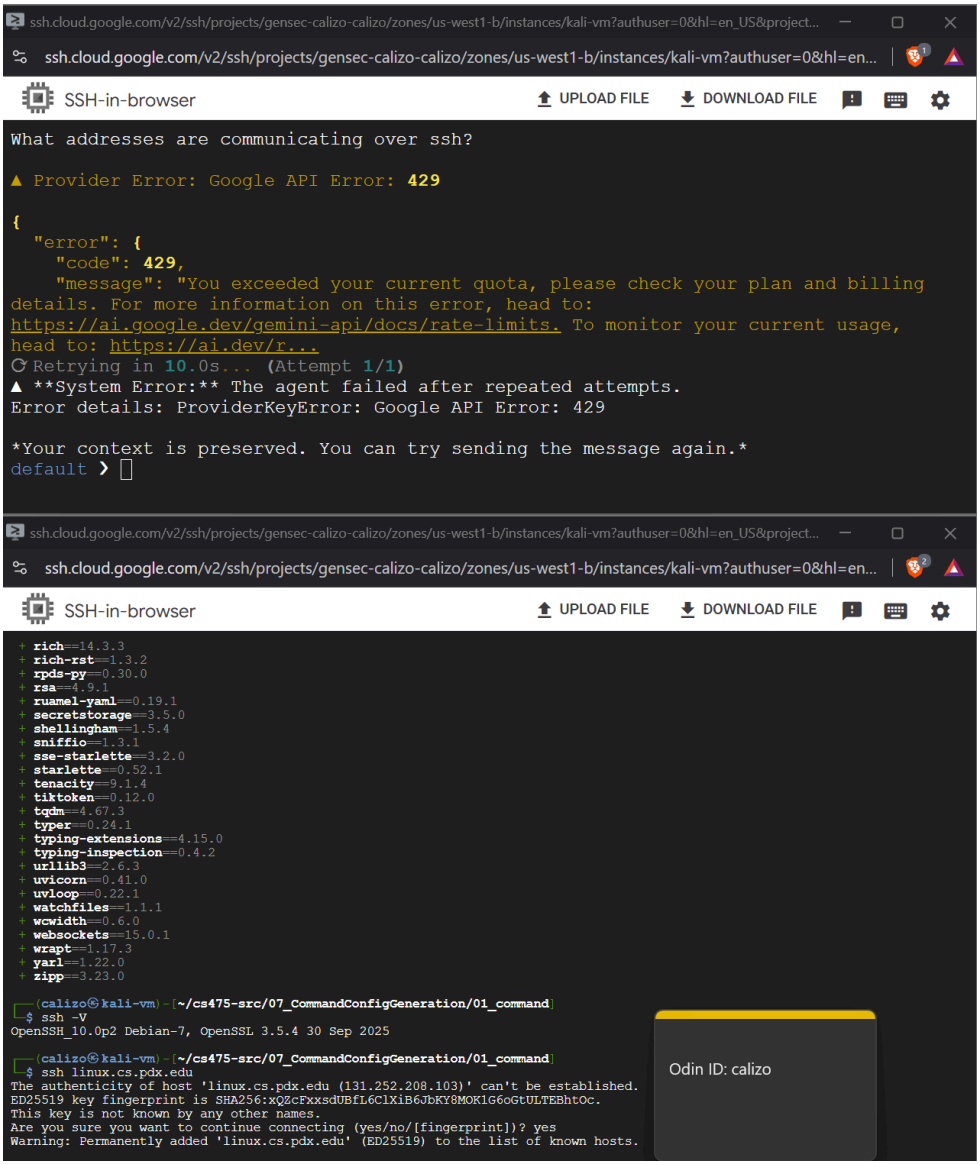
Expected Output:

- Destination host: oregonctf.org
- Destination IP (A record): 35.233.233.233
- Destination port/proto (HTTPS default): 443/tcp

tcpdump:

- IP 35.199.165.186.<ephemeral> > 35.233.233.233.443: Flags [S] ...

- IP 35.233.233.233.443 > 35.199.165.186.<ephemeral>: Flags [S.] ...
- IP 35.199.165.186.<ephemeral> > 35.233.233.233.443: Flags [.] ...
- IP 35.199.165.186.<ephemeral> > 35.233.233.233.443: Flags [P.] ... TLS Client Hello (SNI: oregonctf.org)



Expected output:

- 35.199.165.186 <-> 131.252.208.103 (TCP/22)
  - 35.199.165.186 is my Kali VM
    - linux.cs.pdx.edu resolved to 131.252.208.103
    - SSH traffic is to destination port 22/tcp
- tcpdump might've looked like
  - IP 35.199.165.186.<ephemeral_src_port> > 131.252.208.103.22 ...
  - IP 131.252.208.103.22 > <your_kali_vm_ip>.<ephemeral_src_port> ...

Step 7. Linux package vulnerabilities

1. Include a table of successful commands generated for each task for your notebook

Task	Agent Prompt	Tool Call Command(s) Ran
identify ssh server	Identify the SSH server/service running on this machine (service name and package)	sudo systemctl list-units --type=service --all "ssh*" sudo systemctl list-unit-files --type=service "ssh*" sudo systemctl status ssh.service sudo systemctl status sshd.service sudo systemctl show ssh.service -p FragmentPath -p ExecStart sudo systemctl show sshd.service -p FragmentPath -p ExecStart sudo dpkg-query -S <FRAGMENT_PATH_FROM_systemctl_show> sudo dpkg-query -W -f='${Package}\t${Version}\n' <PACKAGE_NAME_FROM_dpkg-query_-S>
ssh vulnerabilities	Given the current SSH package and version, check for vulnerabilities or security fixes using apt tooling (for example changelog). Summarize	sudo dpkg-query -W -f='${Package}\t${Version}\n' openssh-server sudo dpkg-query -W -f='${Package}\t${Version}\

	any CVE/security references you find	n' openssh-client sudo apt update sudo apt list --installed openssh-server sudo apt list --upgradable openssh-server sudo apt policy openssh-server sudo apt policy openssh-client sudo apt changelog openssh-server sudo apt changelog openssh-client

Step 8. Linux package vulnerabilities (2)

1. Which vulnerabilities did the custom tools discover that weren't found previously?

When I reviewed my OpenSSH banner - OpenSSH_10.0p2 Debian-7, OpenSSL 3.5.4 - the quick checks were fine, but my custom tooling dug deeper. It surfaced two client-side OpenSSH issues tied to "ProxyCommand": CVE-2025-61984 (control characters in usernames from untrusted sources) and CVE-2025-61985 (a NUL byte in an ssh:// URI), both affecting OpenSSH before 10.1 - which includes 10.0p2. Those were the vulnerabilities my custom tools discovered that were not found previously, and they become real only when my SSH client uses "ProxyCommand" in a way that can be influenced by untrusted input. On the crypto side, OpenSSL 3.5.4 also falls in the affected range for CVE-2025-15467, where parsing CMS AuthEnvelopedData with crafted parameters can trigger a stack buffer overflow, so that was another new callout. In short, the custom tools added CVE-2025-61984, CVE-2025-61985, and CVE-2025-15467 to my risk knowledge